Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase III · Secuencias y Transformer

Módulo 12

Representar el lenguaje

Tokenización, embeddings y modelos secuenciales

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

Antes de que un gran modelo de lenguaje procese un texto, un programa sin parámetros aprendidos por descenso de gradiente lo convierte en una secuencia de enteros, y una tabla convierte cada entero en un vector. Este módulo estudia esas dos etapas y la historia de los modelos que precedieron al Transformer. Parte de la representación del texto como bytes mediante UTF-8, desarrolla el algoritmo BPE (*Byte Pair Encoding*) y el modelo unigram de SentencePiece, analiza el coste del vocabulario y sus efectos sobre el comportamiento de los modelos, presenta los embeddings estáticos de word2vec y termina con los modelos de n-gramas, las redes recurrentes y las LSTM, cuyas limitaciones motivan la atención del módulo 13.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. codificar a mano caracteres Unicode en UTF-8 y demostrar que esta codificación es un código prefijo que se autosincroniza;
2. ejecutar a mano el entrenamiento y la codificación de BPE, demostrar cuánto acorta cada fusión y probar que la decodificación de un BPE de bytes recupera el texto original;
3. definir el modelo unigram, demostrar la corrección del algoritmo de Viterbi para segmentar y describir su entrenamiento por EM y poda;
4. calcular el coste en parámetros de un vocabulario y la compresión de un tokenizador, y explicar los efectos de la tokenización sobre el recuento de letras, la aritmética y los idiomas;
5. definir los embeddings, la similitud coseno y la información mutua puntual, derivar los gradientes de skip-gram con softmax y con muestreo negativo y relacionar este último con la PMI;
6. analizar la dispersión de los n-gramas, demostrar la cota exponencial del gradiente en una red recurrente, describir la LSTM y el recorte del gradiente, y justificar por qué estos modelos no escalan como el Transformer.

Requisitos previos

Del módulo 1 se usan los tokens, el tokenizador como par codificación-decodificación, los modelos de n-gramas y la entropía cruzada; del módulo 2, el producto escalar, la norma y la similitud coseno; del módulo 3, los valores y vectores propios de una matriz simétrica y la norma espectral; del módulo 4, la regla de la cadena multivariable y el jacobiano; del módulo 6, la información mutua; del módulo 8, la regresión logística y la softmax con entropía cruzada; del módulo 9, el modelo de lenguaje neuronal de Bengio *et al.*, y del módulo 11, el

desvanecimiento del gradiente y las conexiones residuales. Se usan también la numeración binaria y hexadecimal.

Organización

Tema	Contenido	Fuentes principales
1	Del texto a los bytes: Unicode y UTF-8	[10], [22], [4]
2	El algoritmo BPE	[20], [10], [4], [19]
3	El modelo unigram, SentencePiece y el vocabulario	[11], [12], [10], [18]
4	Embeddings y word2vec	[10], [3], [16], [13]
5	Modelos secuenciales: n-gramas, RNN y LSTM	[10], [3], [2], [17], [9]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: $\star$ comprensión, $\star\star$ cálculo y $\star\star\star$ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. La parte de tokenización sigue el capítulo 2 de Jurafsky y Martin [10], «Words and Tokens» (§2.3 «Unicode» y §2.4 «Subword Tokenization: Byte-Pair Encoding»), y la sección 2 de la tarea 1 del curso CS336 de Stanford [4], en la que se programa un tokenizador BPE de bytes. Los algoritmos se toman de sus artículos originales: Sennrich, Haddow y Birch [20] para BPE, Radford *et al.* [19] para su versión a nivel de byte, Kudo [11] para el modelo unigram y Kudo y Richardson [12] para SentencePiece; la codificación UTF-8 se define en el RFC 3629 [22]. Los embeddings siguen el capítulo 5 de [10], «Embeddings», y la clase 2 de CS224N [3], «Word Vectors», con los artículos de Mikolov *et al.* [15, 16] y de Levy y Goldberg [13]. Los modelos recurrentes siguen el capítulo 14 de [10], «RNNs and LSTMs», la clase 4 de CS224N, el capítulo 10 de Goodfellow, Bengio y Courville [8] y los capítulos 9 y 10 de Zhang *et al.* [23]; los resultados sobre el gradiente proceden de Bengio, Simard y Frasconi [2] y de Pascanu, Mikolov y Bengio [17], y la LSTM, de Hochreiter y Schmidhuber [9]. Sobre los modelos comerciales (Claude, Gemini, Grok) solo se usa información pública: el informe técnico de Gemini [6] indica que su tokenizador es de tipo SentencePiece, los pesos de Grok-1 publicados por xAI incluyen su vocabulario, y Anthropic no ha publicado los detalles del tokenizador de Claude.

Notación

Símbolo	Significado
Σ, Σ^*	alfabeto de caracteres (puntos de código Unicode) y cadenas sobre él
$\mathcal{B} = \{0, \ldots, 255\}$	conjunto de los bytes; $\mathcal{B}^*$, cadenas de bytes
`U+00F1`, `C3`, `110xxxxx`	punto de código, byte en hexadecimal y patrón de bits
$\mathcal{V}$, $K = \lvert\mathcal{V}\rvert$, $\mathcal{V}_0$	vocabulario, su tamaño y vocabulario inicial de BPE
k	número de fusiones de BPE
$u \Vert v$	concatenación de las cadenas u y v
$s = (t_1, \ldots, t_m)$, $p(t)$	segmentación en tokens y probabilidad unigram de un token
$E \in \mathbb{R}^{K \times d}$, e_i	matriz de embeddings y vector *one-hot* del token i
v_w, u_w	vector de una palabra como centro y como contexto en skip-gram
σ, $\odot$	sigmoide logística y producto componente a componente
h_t, c_t	estado oculto de una red recurrente y celda de memoria de una LSTM
f_t, i_t, o_t	puertas de olvido, de entrada y de salida de una LSTM
$\lVert\cdot\rVert$	norma euclídea de un vector o norma espectral de una matriz

Índice

Presentación del módulo **ii**

1 Del texto a los bytes: Unicode y UTF-8 **1**

1.1 El problema de la tokenización . . . 1
1.2 Unicode y puntos de código . . . 1
1.3 La codificación UTF-8 . . . 2
1.4 Propiedades de UTF-8 . . . 3
1.5 Consecuencias para el español y para los LLM . . . 4
Ejercicios del tema 1 . . . 4

2 El algoritmo BPE **6**

2.1 Entrenamiento . . . 6
2.2 Codificación y decodificación . . . 8
2.3 Desempates y determinismo . . . 8
2.4 BPE a nivel de byte y pretokenización . . . 8
Ejercicios del tema 2 . . . 9

3 El modelo unigram, SentencePiece y el vocabulario **11**

3.1 El modelo unigram . . . 11
3.2 Segmentación óptima: el algoritmo de Viterbi . . . 12
3.3 Entrenamiento por EM y poda . . . 12
3.4 SentencePiece . . . 13
3.5 El tamaño del vocabulario . . . 13
3.6 Efectos de la tokenización . . . 14
Ejercicios del tema 3 . . . 15

4 Embeddings y word2vec **16**

4.1 La matriz de embeddings . . . 16
4.2 La hipótesis distribucional . . . 16
4.3 Skip-gram . . . 17
4.4 Muestreo negativo . . . 18
4.5 Muestreo negativo y PMI . . . 18
4.6 Embeddings en un LLM . . . 19
Ejercicios del tema 4 . . . 20

5 Modelos secuenciales: n-gramas, RNN y LSTM **21**
5.1 N-gramas y dispersión . . . 21
5.2 Redes recurrentes . . . 21
5.3 Retropropagación en el tiempo y desvanecimiento del gradiente . . . 22
5.4 La LSTM . . . 24
5.5 Secuencialidad y el paso al Transformer . . . 24
Ejercicios del tema 5 . . . 25

A Soluciones de los ejercicios **27**
A.1 Tema 1 . . . 27
A.2 Tema 2 . . . 29
A.3 Tema 3 . . . 32
A.4 Tema 4 . . . 35
A.5 Tema 5 . . . 38

Bibliografía **41**

Tema 1

Del texto a los bytes: Unicode y UTF-8

Un tokenizador transforma texto en una secuencia de enteros. Antes de elegir las unidades conviene fijar qué es un texto para un ordenador: una sucesión de puntos de código Unicode que se almacena como una sucesión de bytes. Este tema plantea el compromiso entre longitud de la secuencia y tamaño del vocabulario, define la codificación UTF-8 y demuestra las dos propiedades que la hacen adecuada como punto de partida de los tokenizadores modernos: es un código prefijo y se autosincroniza. Se sigue [10, §2.3], la definición del RFC 3629 [22] y la §2 de la tarea 1 de CS336 [4].

1.1 El problema de la tokenización

Se recuerda del módulo 1 que un *tokenizador* es un par de aplicaciones $\tau\colon \Sigma^* \to \mathcal{V}^*$ y $\delta\colon \mathcal{V}^* \to \Sigma^*$ con $\delta(\tau(s)) = s$ para toda cadena s. La elección de $\mathcal{V}$ fija la granularidad y, con ella, dos magnitudes que compiten: el tamaño $K = |\mathcal{V}|$ y la longitud $|\tau(s)|$ de las secuencias.

Cuadro 1.1: Granularidad de la tokenización, con la palabra «caballerías» como ejemplo.

Unidad	Tokens	K típico	Limitación principal
Carácter	11	$\sim 10^2$	secuencias largas; la atención cuesta $O(n^2)$
Palabra	1	$> 5 \cdot 10^5$	palabras fuera del vocabulario; formas sin relación
Subpalabra	2–5	$3{,}2 \cdot 10^4$–$2{,}6 \cdot 10^5$	hay que entrenar el tokenizador

Con un vocabulario de palabras, «caballería» y «caballerías» son tokens sin ninguna relación, y una palabra nueva o con una errata no tiene representación. Con caracteres, cualquier texto es representable, pero las secuencias son largas, y el coste de la atención crece con el cuadrado de la longitud (módulo 13). Las subpalabras reparten el vocabulario según la frecuencia: las palabras comunes son un solo token y las raras se componen de piezas frecuentes.

Observación 1.1. El tokenizador se entrena antes que el modelo y por separado, con un algoritmo de conteo, y después se congela. El modelo solo recibe los enteros que produce. En los temas 2 y 3 se estudian los dos algoritmos de entrenamiento más usados.

1.2 Unicode y puntos de código

Definición 1.2 (Punto de código). Unicode asigna a cada carácter un entero, su *punto de código*, del conjunto $\{0, 1, \ldots, \texttt{10FFFF}_{16}\}$, que se escribe en hexadecimal con el prefijo `U+`. Los puntos de código del intervalo `U+D800`–`U+DFFF` (*sustitutos*) están reservados y no representan caracteres. Los restantes se llaman *valores escalares Unicode* [22].

Por ejemplo, «a» es `U+0061` (97), «ñ» es `U+00F1` (241) y el signo del euro es `U+20AC` (8364). Un punto de código es abstracto: no fija la forma gráfica (el glifo) con que se dibuja el carácter [10, §2.3.1]. Unicode 16.0 define más de $1{,}5 \cdot 10^5$ caracteres.

Un modelo podría usar los puntos de código como vocabulario inicial, pero este tendría más de $1{,}3 \cdot 10^5$ símbolos, casi todos raros. Por eso los tokenizadores modernos parten de la representación del texto en bytes.

1.3 La codificación UTF-8

Definición 1.3 (UTF-8). La codificación *UTF-8* asigna a cada valor escalar c una cadena de 1 a 4 bytes según el cuadro 1.2. Los bits de c, escritos en binario con el número de bits útiles de su fila y completados con ceros a la izquierda, rellenan de izquierda a derecha las posiciones marcadas con `x`. La codificación de un texto es la concatenación de las codificaciones de sus caracteres.

Cuadro 1.2: Definición de UTF-8 [22]. Se llama *byte inicial* al primero de cada fila y *bytes de continuación* a los demás.

Puntos de código	Bytes	Bits útiles	Patrón de bits
`U+0000`–`U+007F`	1	7	`0xxxxxxx`
`U+0080`–`U+07FF`	2	11	`110xxxxx 10xxxxxx`
`U+0800`–`U+FFFF`	3	16	`1110xxxx 10xxxxxx 10xxxxxx`
`U+10000`–`U+10FFFF`	4	21	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

Se exige además que cada carácter use la fila más corta en la que cabe: las codificaciones *sobrelargas*, como escribir «a» con dos bytes, no son UTF-8 válido. Como consecuencia, los bytes `C0`, `C1` y `F5`–`FF` no aparecen nunca en un texto UTF-8 [22].

Ejemplo 1.4 (La letra ñ). El punto de código de «ñ» es $\texttt{F1}_{16} = 241 > 127$, luego usa la fila de 2 bytes. En binario con 11 bits, $241 = \texttt{00011110001}$, que se reparte en $5 + 6$ bits: `00011` y `110001`. Los bytes son

$$\texttt{110}\,\texttt{00011} = \texttt{11000011} = \texttt{C3}_{16} = 195, \qquad \texttt{10}\,\texttt{110001} = \texttt{10110001} = \texttt{B1}_{16} = 177.$$

La figura 1.1 muestra el reparto. Todas las vocales con tilde del español tienen puntos de código entre `U+00C0` y `U+00FF`, de modo que su byte inicial es `C3`: «á» = `C3 A1`, «é» = `C3 A9`, «ó» = `C3 B3`.

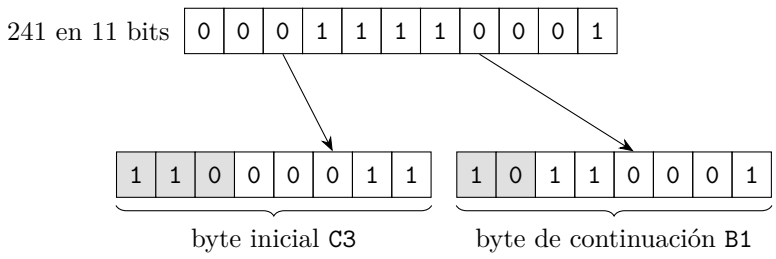


Figura 1.1: Codificación UTF-8 de «ñ» (ejemplo 1.4). Las casillas sombreadas son los bits fijos del patrón; las blancas, los bits útiles del punto de código.

Proposición 1.5. *El número de puntos de código que UTF-8 codifica con 1, 2, 3 y 4 bytes es, respectivamente,* 128*,* 1920*,* 63 488 *y* 1 048 576*. Descontando los* 2048 *sustitutos, que pertenecen a la fila de 3 bytes, hay* 1 112 064 *valores escalares.*

Demostración. Cada fila del cuadro 1.2 cubre un intervalo de enteros; su cardinal es la diferencia de los extremos más uno: $\texttt{80}_{16} = 128$; $\texttt{800}_{16} - \texttt{80}_{16} = 2048 - 128 = 1920$; $\texttt{10000}_{16} - \texttt{800}_{16} = 65\,536 - 2048 = 63\,488$; $\texttt{110000}_{16} - \texttt{10000}_{16} = 1\,114\,112 - 65\,536 = 1\,048\,576$. Los sustitutos son $\texttt{E000}_{16} - \texttt{D800}_{16} = 2048$, todos en el intervalo de 3 bytes. El total es $1\,114\,112 - 2048 = 1\,112\,064$. ∎

1.4 Propiedades de UTF-8

Definición 1.6 (Código prefijo). Sea $C\colon \Sigma \to \mathcal{B}^*$ una aplicación que asigna a cada símbolo una cadena de bytes no vacía. C es un *código prefijo* si ninguna palabra del código $C(a)$ es prefijo propio de otra $C(b)$ y, además, C es inyectiva.

Teorema 1.7 (UTF-8 es un código prefijo). *La codificación UTF-8 de un carácter es inyectiva y ninguna palabra del código es prefijo propio de otra. En consecuencia, su extensión a cadenas,* $\mathrm{enc}\colon \Sigma^* \to \mathcal{B}^*$*, es inyectiva: toda cadena de bytes UTF-8 válida se descompone de forma única en caracteres.*

Demostración. Paso 1 (el byte inicial determina la longitud). El byte inicial de una palabra de longitud n empieza por `0` si $n = 1$ y por n unos seguidos de un cero si $n \in \{2, 3, 4\}$. Estos cuatro prefijos de bits son incompatibles entre sí, de modo que la longitud de una palabra del código se lee en su primer byte.

Paso 2 (sin prefijos propios). Si $C(a)$ fuera prefijo propio de $C(b)$, ambas palabras tendrían el mismo primer byte y, por el paso 1, la misma longitud, lo que contradice que el prefijo sea propio.

Paso 3 (inyectividad por caracteres). Dentro de una fila, la aplicación que lleva los bits útiles a la cadena de bytes es una biyección entre enteros de ℓ bits y cadenas con el patrón fijado. Si $C(a) = C(b)$, ambas están en la misma fila (paso 1) y tienen los mismos bits útiles, luego $a = b$.

Paso 4 (cadenas). Sea $b_1 \dots b_L = \mathrm{enc}(x) = \mathrm{enc}(y)$. El primer byte fija la longitud n de la primera palabra (paso 1); los n primeros bytes son, a la vez, $C(x_1)$ y $C(y_1)$, luego $x_1 = y_1$ (paso 3). Se elimina esa palabra y se razona por inducción sobre L. ∎

Proposición 1.8 (Autosincronización). *En una cadena UTF-8 válida, un byte es de continuación si y solo si sus dos bits más altos son* `10`*. Por tanto, desde cualquier posición se encuentra el comienzo del carácter que la contiene retrocediendo a lo sumo 3 bytes.*

Demostración. Por el cuadro 1.2, todo byte de continuación empieza por `10`, y ningún byte inicial lo hace: los iniciales empiezan por `0`, `110`, `1110` o `11110`. Basta, pues, retroceder mientras el byte empiece por `10`; como un carácter tiene a lo sumo 3 bytes de continuación, se retrocede como mucho 3 posiciones. ∎

La autosincronización tiene una consecuencia práctica: si un archivo se corta por la mitad de un carácter, solo se pierde ese carácter, y los siguientes se decodifican bien [10, §2.3.2].

Además, UTF-8 es compatible con ASCII, porque los 128 primeros puntos de código ocupan un byte con su valor ASCII.

1.5 Consecuencias para el español y para los LLM

Observación 1.9. Las letras con tilde, la diéresis, la «ñ» y los signos de apertura «¿» y «¡» y las comillas angulares ocupan 2 bytes. En el fragmento inicial de *El Quijote* que se usa en la práctica del módulo hay 2484 caracteres y 2536 bytes; los 52 bytes de más corresponden a 48 letras con tilde o «ñ» y a 4 comillas angulares. La palabra «año» tiene 3 caracteres y 4 bytes.

Corolario 1.10. *Todo texto se representa sin pérdida como una cadena de $\mathcal{B}^*$, con $|\mathcal{B}| = 2^8 = 256$. Un tokenizador cuyo vocabulario inicial sean los 256 bytes nunca necesita un token «desconocido».*

Demostración. Por el teorema 1.7, enc es inyectiva, luego existe una inversa por la izquierda enc^{-1} sobre su imagen y $\text{enc}^{-1}(\text{enc}(x)) = x$. Cada byte es uno de los 2^8 valores de $\mathcal{B}$, todos presentes en el vocabulario inicial. ∎

El precio es que un carácter no ASCII ocupa varios tokens iniciales, y un tokenizador de bytes debe aprender fusiones como `C3` + `B1` → «ñ» antes de poder formar «año». Además, un token puede contener solo una parte de un carácter; el tema 2 analiza qué ocurre entonces al decodificar.

Lecturas recomendadas para este tema

- Jurafsky y Martin [10], §2.3 «Unicode», con §2.3.1 «Code Points» y §2.3.2 «UTF-8 Encoding».
- Yergeau, RFC 3629 [22]: definición normativa de UTF-8.
- Stanford CS336 [4], tarea 1, §2.1 y §2.2 (Unicode y codificaciones Unicode).

Ejercicios del tema 1

Ejercicio 1.1 ⋆ Un programa decodifica una cadena de bytes UTF-8 decodificando cada byte por separado y concatenando los resultados. (a) Dé una palabra española para la que falla y explique por qué. (b) Dé una cadena de 2 bytes que no corresponda a ningún carácter y justifíquelo.

Fuente: adaptado de Stanford CS336 [4], tarea 1, §2.2, problema *unicode2*, apartados (b) y (c).

Ejercicio 1.2 ⋆⋆ La «é» es `U+00E9`. Obtenga sus bytes UTF-8 en binario y en hexadecimal.

Ejercicio 1.3 ⋆⋆ ¿Cuántos bytes UTF-8 ocupan «pingüino», «Ñandú» y «café» seguido de un espacio y del carácter `U+2615` (taza de café)? Justifíquelo carácter a carácter.

Ejercicio 1.4 ⋆⋆ Codifique a mano en UTF-8, mostrando los bits, el signo «¿» (`U+00BF`) y el carácter chino `U+8FDB`. ¿Cuántos bytes ocupa cada uno?

Fuente: adaptado de Jurafsky y Martin [10], cap. 2, ejercicio 2.3 (se sustituye la «é» por «¿»).

Ejercicio 1.5 ⋆⋆ Compruebe la proposición 1.5 y calcule qué fracción de los valores escalares Unicode se codifica con 4 bytes. ¿Por qué el emoji `U+1F600` ocupa 4 bytes?

Ejercicio 1.6 ⋆⋆⋆ (a) Demuestre que, si se admitieran codificaciones sobrelargas, la decodificación dejaría de ser inyectiva: habría cadenas de bytes distintas que representan el mismo texto. Explique por qué esto perjudica a un tokenizador de bytes. (b) Demuestre que los bytes `C0` y `C1` solo podrían aparecer en una codificación sobrelarga.

Tema 2

El algoritmo BPE

El algoritmo BPE (*Byte Pair Encoding*) construye un vocabulario de subpalabras de abajo arriba: parte de los símbolos básicos y fusiona, una y otra vez, el par adyacente más frecuente. Lo propuso Gage como método de compresión de datos [5], y Sennrich, Haddow y Birch lo adaptaron a la segmentación de palabras para la traducción automática [20, §3.2]; Radford *et al.* lo aplicaron a bytes en GPT-2 [19, §2.2]. Este tema define el algoritmo, demuestra cuánto acorta cada fusión y prueba que la decodificación recupera el texto. Se sigue [10, §2.4] y la §2.4 de la tarea 1 de CS336 [4].

2.1 Entrenamiento

Definición 2.1 (Corpus de palabras y pares). Un *corpus de entrenamiento* es un conjunto finito de palabras w, cada una con una frecuencia $n_w \geq 1$ y escrita como una secuencia de símbolos de un vocabulario $\mathcal{V}$. Para dos símbolos $a, b \in \mathcal{V}$, la *frecuencia del par* (a, b) es

$$N(a,b) = \sum_{w} n_w \cdot \#\{\, i : w_i = a,\ w_{i+1} = b \,\},$$

el número de posiciones en que b sigue inmediatamente a a, ponderado por la frecuencia de cada palabra.

Definición 2.2 (Fusión). La *fusión* del par (a, b) sustituye en cada palabra, recorriéndola de izquierda a derecha, cada aparición de a seguido de b por un símbolo nuevo $ab = a\|b$, y añade ab al vocabulario.

Definición 2.3 (Entrenamiento de BPE). Dados un corpus escrito en un vocabulario inicial $\mathcal{V}_0$ y un número de fusiones k, el *entrenamiento de BPE* repite k veces:

1. calcular $N(a, b)$ para todos los pares adyacentes;
2. elegir un par (a^*, b^*) de frecuencia máxima, con una regla de desempate fija;
3. fusionarlo y anotar la regla $a^* + b^* \to a^* b^*$.

El resultado es la *lista ordenada de fusiones* $\mu_1, \ldots, \mu_k$, que constituye el tokenizador.

Las palabras se obtienen del texto en un paso previo, la *pretokenización*, de modo que las fusiones nunca cruzan el límite entre dos palabras [20, §3.2]. Sennrich *et al.* añaden al final de cada palabra un símbolo especial de fin de palabra para poder reconstruir los espacios.

Proposición 2.4. *Tras k fusiones, el vocabulario tiene a lo sumo $|\mathcal{V}_0| + k$ símbolos, con igualdad cuando las k cadenas creadas son distintas entre sí y no pertenecen a $\mathcal{V}_0$. En particular, si $\mathcal{V}_0$ está formado por símbolos de longitud 1, $|\mathcal{V}| = |\mathcal{V}_0| + k$ salvo repeticiones de una misma cadena obtenida por dos caminos distintos.*

Demostración. Cada fusión añade exactamente un símbolo, $a^*\|b^*$, y no elimina ninguno; por inducción sobre k, el número de símbolos añadidos es k y el vocabulario tiene a lo sumo $|\mathcal{V}_0| + k$ elementos distintos. Si los k símbolos creados son cadenas distintas y ajenas a $\mathcal{V}_0$, no hay coincidencias y se alcanza la igualdad. Un símbolo creado tiene longitud al menos 2, de modo que nunca coincide con un símbolo de longitud 1. ■

En la práctica se usa la igualdad $|\mathcal{V}| = |\mathcal{V}_0| + k$, y k es el único hiperparámetro del entrenamiento [20, §3.2].

Lema 2.5. *Si $a \neq b$, dos apariciones del par (a, b) en una secuencia no comparten ninguna posición.*

Demostración. Dos apariciones distintas que compartieran una posición ocuparían las posiciones $(i, i+1)$ y $(i+1, i+2)$. El símbolo de la posición $i+1$ sería el segundo del primer par, luego b, y el primero del segundo, luego a. Así $a = b$, en contra de la hipótesis. ■

Proposición 2.6 (Compresión de una fusión). *Sea un corpus con L símbolos en total (contando cada palabra n_w veces) y sea (a, b), con $a \neq b$, un par de frecuencia $N(a, b) = c$. Tras fusionarlo, el corpus tiene $L - c$ símbolos.*

Demostración. Por el lema 2.5, las c apariciones ocupan posiciones disjuntas, de modo que la fusión las sustituye todas. Cada sustitución convierte dos símbolos en uno y resta exactamente 1 a la longitud; las sustituciones no interfieren entre sí, así que la longitud total baja en c. ■

Observación 2.7. Si $a = b$, las apariciones pueden solaparse: «`aaa`» contiene dos pares $(\mathtt{a}, \mathtt{a})$, pero la fusión de izquierda a derecha solo puede aplicar una y deja «`aa·a`». En ese caso la longitud baja en el número de apariciones *no solapadas*. Por la proposición 2.6, elegir el par más frecuente es elegir la fusión que más acorta el corpus en ese paso: BPE es un algoritmo *voraz*. No garantiza la segmentación globalmente más corta con k fusiones, pero funciona bien en la práctica.

Ejemplo 2.8 (Entrenamiento a mano). Corpus: «casa» ×3, «casas» ×2, «masa» ×1, «mesa» ×1, con $\mathcal{V}_0$ formado por las letras y $L = 3 \cdot 4 + 2 \cdot 5 + 4 + 4 = 30$ símbolos. En el primer paso, $(\mathtt{a}, \mathtt{s})$ aparece una vez en cada «casa», dos en cada «casas», una en «masa» y ninguna en «mesa»: $N(\mathtt{a}, \mathtt{s}) = 3 + 4 + 1 = 8$. El cuadro 2.1 recoge los cuatro primeros pasos. Cada longitud es la anterior menos la frecuencia del par fusionado, como afirma la proposición 2.6: $30 - 8 = 22$, $22 - 5 = 17$, $17 - 3 = 14$ y $14 - 2 = 12$.

Cuadro 2.1: Cuatro fusiones de BPE sobre el corpus del ejemplo 2.8. El punto separa los símbolos de cada palabra.

Paso	Pares más frecuentes	Fusión	Corpus resultante	L
0	—	—	`c·a·s·a` (×3); `c·a·s·a·s` (×2); `m·a·s·a`; `m·e·s·a`	30
1	(a, s) 8, (s, a) 7	a + s	`c·as·a` (×3); `c·as·as` (×2); `m·as·a`; `m·e·s·a`	22
2	(c, as) 5, (as, a) 4	c + as	`cas·a` (×3); `cas·as` (×2); `m·as·a`; `m·e·s·a`	17
3	(cas, a) 3, (cas, as) 2	cas + a	`casa` (×3); `cas·as` (×2); `m·as·a`; `m·e·s·a`	14
4	(cas, as) 2, resto 1	cas + as	`casa` (×3); `casas` (×2); `m·as·a`; `m·e·s·a`	12

2.2 Codificación y decodificación

Definición 2.9 (Codificación BPE). Para codificar una palabra nueva se escribe en símbolos de $\mathcal{V}_0$ y se aplican las fusiones $\mu_1, \ldots, \mu_k$ *en el orden en que se aprendieron*, cada una a toda la palabra, de izquierda a derecha. La *decodificación* de una secuencia de tokens es la concatenación de las cadenas que representan.

Ejemplo 2.10. Con las fusiones del cuadro 2.1, «pasas» $=$ `p·a·s·a·s` pasa a `p·as·as` con μ_1 y ninguna otra fusión aplica: 3 tokens. «cosas» da `c·o·s·as`, 4 tokens, porque el par (`c`, `o`) nunca se vio. «casasas» da `casas·as`, 2 tokens. Las frecuencias del texto que se codifica no intervienen: solo importa el orden de las fusiones [10, §2.4.2].

Teorema 2.11 (Corrección de BPE). *Sea $\mathcal{V}_0$ un vocabulario inicial tal que toda cadena admite una escritura en símbolos de $\mathcal{V}_0$, y sea δ la decodificación por concatenación. Para toda cadena x, la codificación $\tau(x)$ está definida y $\delta(\tau(x)) = x$.*

Demostración. Sea $t^{(j)} = (t_1^{(j)}, \ldots, t_{m_j}^{(j)})$ la secuencia de tokens tras aplicar las j primeras fusiones, y sea $\delta(t^{(j)}) = t_1^{(j)} \| \ldots \| t_{m_j}^{(j)}$. Se demuestra por inducción sobre j que $\delta(t^{(j)}) = x$.

Para $j = 0$, $t^{(0)}$ es la escritura de x en símbolos de $\mathcal{V}_0$, que existe por hipótesis, y su concatenación es x. Supóngase $\delta(t^{(j-1)}) = x$. La fusión $\mu_j = (a, b)$ sustituye ciertos pares de tokens consecutivos $t_i = a$, $t_{i+1} = b$ por el token $a\|b$; como la concatenación es asociativa, $\ldots \|a\|b\| \cdots = \ldots \|(a\|b)\| \ldots$ y la concatenación total no cambia. Así $\delta(t^{(j)}) = x$ para todo j, y en particular para $j = k$. ∎

Corolario 2.12. *Un tokenizador BPE nunca falla: en el peor caso no se aplica ninguna fusión y se devuelven los símbolos de $\mathcal{V}_0$. Con $\mathcal{V}_0 = \mathcal{B}$ (BPE a nivel de byte), por el corolario 1.10 no hay tokens desconocidos.*

2.3 Desempates y determinismo

Cuando varios pares empatan en frecuencia máxima, el paso 2 de la definición 2.3 necesita una regla. Cualquier regla sirve, pero debe ser fija para que el tokenizador sea reproducible. La tarea de CS336 elige el par mayor en orden lexicográfico [4, §2.4]; otra opción frecuente es el primer par que aparece en el corpus. Reglas distintas producen tokenizadores distintos (ejercicio 2.5).

Observación 2.13. La codificación de la definición 2.9 es determinista: la misma cadena da siempre la misma secuencia de tokens. El modelo unigram del tema 3, en cambio, define una distribución sobre segmentaciones y permite muestrearlas.

2.4 BPE a nivel de byte y pretokenización

GPT-2 aplica BPE a los bytes UTF-8 del texto, con $\mathcal{V}_0 = \mathcal{B}$ y $|\mathcal{V}_0| = 256$, en lugar de los más de $1{,}3 \cdot 10^5$ símbolos que exigiría partir de los puntos de código [19, §2.2]. Su vocabulario tiene 50 257 tokens [19, §2.3]: los 256 bytes, un token especial de fin de texto y 50 000 fusiones (ejercicio 2.2).

Definición 2.14 (Pretokenización). La *pretokenización* divide el texto en fragmentos mediante una expresión regular antes de aplicar BPE, y las fusiones se aplican dentro de cada

fragmento [10, §2.6.9]. En GPT-2, cada fragmento es típicamente una palabra con su espacio delante (␣`lugar`, ␣`de`), un signo de puntuación o un grupo de dígitos; así se impide fusionar bytes de categorías distintas [19, §2.2].

Sin pretokenización, BPE aprende tokens que mezclan palabras y puntuación, como «`de␣su␣hacienda.␣`». Una consecuencia de la pretokenización es que ␣`hidalgo` y `hidalgo` son tokens distintos: un espacio de más al final de un texto cambia su tokenización.

Observación 2.15 (Tokens que contienen medio carácter). Con $\mathcal{V}_0 = \mathcal{B}$, un token puede ser un fragmento de un carácter multibyte, por ejemplo el byte `C3` solo. Por el teorema 2.11, la concatenación de todos los tokens de un texto es su cadena UTF-8, que se decodifica sin pérdida (teorema 1.7). Pero un token aislado puede no ser UTF-8 válido, y una interfaz que decodifica cada token al recibirlo debe guardar en un búfer los bytes de un carácter incompleto (ejercicio 2.7).

Lecturas recomendadas para este tema

- Sennrich, Haddow y Birch [20], §3.2 «Byte Pair Encoding (BPE)», con el algoritmo 1.
- Jurafsky y Martin [10], §2.4 «Subword Tokenization: Byte-Pair Encoding» (§2.4.1–2.4.3) y §2.6.9 sobre las expresiones regulares de la pretokenización.
- Stanford CS336 [4], tarea 1, §2.4 (entrenamiento de BPE) y §2.5 (implementación).
- Radford *et al.* [19], §2.2 «Input Representation».
- Zhang *et al.* [23], §15.6.2 «Byte Pair Encoding».

Ejercicios del tema 2

Ejercicio 2.1 ⋆ ¿Por qué el BPE a nivel de byte nunca produce un token desconocido? ¿Qué tamaño tiene su vocabulario inicial?

Ejercicio 2.2 ⋆ (a) Si BPE parte de n símbolos, ¿cuántas fusiones hacen falta para llegar a un vocabulario de m símbolos? (b) GPT-2 parte de 256 bytes, tiene un token especial de fin de texto, que no se obtiene por fusión, y su vocabulario mide 50 257. ¿Cuántas fusiones aprendió?

Fuente: adaptado de Zhang *et al.* [23], §15.6, ejercicio 3 (se añade el apartado b).

Ejercicio 2.3 ⋆⋆ Aplique 3 fusiones de BPE al corpus «rana» ×1, «lana» ×2, «ana» ×1, «plan» ×1. Indique en cada paso el par elegido, su frecuencia y el número total de símbolos.

Ejercicio 2.4 ⋆⋆ Con las cuatro fusiones del cuadro 2.1, codifique «masas», «casa» y «cascas».

Ejercicio 2.5 ⋆⋆ Ante un empate de frecuencias se fusiona el par lexicográficamente mayor (se compara el primer símbolo y, si coincide, el segundo; una cadena es mayor que cualquiera de sus prefijos propios). Corpus: «paso» ×4, «pasos» ×2, «piso» ×3, «pisos» ×3. (a) Aplique 4 fusiones indicando conteos, empates, par elegido y número de símbolos. (b) Codifique «peso», «sopa» y «oso».

Fuente: adaptado de Stanford CS336 [4], tarea 1, §2.4, ejemplo *bpe_example* (corpus cambiado).

Ejercicio 2.6 ⋆⋆⋆ Demuestre la proposición 2.6 sin usar el lema 2.5, razonando directamente sobre las posiciones. Dé un contraejemplo cuando $a = b$ y calcule la longitud tras fusionar $(\texttt{a}, \texttt{a})$ en «`aaaa`» y en «`aaaaa`».

Ejercicio 2.7 ⋆⋆⋆ Un tokenizador BPE de bytes tiene como token el byte `C3` solo. (a) ¿Qué ocurre al decodificarlo aislado? (b) Demuestre que decodificar la secuencia completa de tokens de un texto devuelve siempre el texto original. (c) Explique qué ve un usuario si una interfaz decodifica cada token al recibirlo y la «ñ» llega partida en dos tokens, y proponga un remedio.

Tema 3

El modelo unigram, SentencePiece y el vocabulario

BPE construye el vocabulario de abajo arriba y codifica de forma determinista. El modelo unigram de Kudo [11] sigue el camino inverso: parte de un vocabulario grande, asigna una probabilidad a cada token, segmenta eligiendo la segmentación más probable y entrena podando. Este tema define el modelo, demuestra la corrección del algoritmo de Viterbi que lo segmenta, describe la biblioteca SentencePiece [12] y estudia el coste del vocabulario y los efectos de la tokenización sobre el comportamiento de los modelos. Se sigue [11, §3.2–3.4], [12, §3] y [10, §2.4.3].

3.1 El modelo unigram

Definición 3.1 (Modelo unigram de segmentación). Sea $\mathcal{V}$ un vocabulario de subpalabras y sea $p\colon \mathcal{V} \to (0,1]$ con $\sum_{t\in\mathcal{V}} p(t) = 1$. Una *segmentación* de una cadena x es una secuencia $s = (t_1, \dots, t_m)$ de tokens de $\mathcal{V}$ con $t_1 \| \dots \| t_m = x$; se denota $S(x)$ el conjunto de segmentaciones de x. El *modelo unigram* asigna a cada segmentación la probabilidad

$$P(s) = \prod_{i=1}^{m} p(t_i),$$

y la segmentación elegida es $s^* = \arg\max_{s\in S(x)} P(s)$ [11, §3.2].

El modelo es el de unigramas del módulo 1, ahora sobre subpalabras: los tokens se suponen independientes. Como cada factor es menor que 1, las segmentaciones con menos tokens tienden a ser más probables, igual que las palabras cortas en un modelo de bigramas.

Ejemplo 3.2. Sea $p(\texttt{casa}) = 0{,}2$, $p(\texttt{a}) = 0{,}2$, $p(\texttt{as}) = 0{,}15$, $p(\texttt{s}) = 0{,}1$, $p(\texttt{cas}) = 0{,}1$, $p(\texttt{c}) = 0{,}05$ (el resto de la masa corresponde a otros tokens). El cuadro 3.1 enumera las siete segmentaciones de «casas». La más probable es $s^* = (\texttt{casa}, \texttt{s})$, con $P(s^*) = 0{,}02$.

Cuadro 3.1: Segmentaciones de «casas» con el vocabulario del ejemplo 3.2.

Segmentación	Producto	$P(s)$
casa · s	$0{,}2 \cdot 0{,}1$	0,02
cas · as	$0{,}1 \cdot 0{,}15$	0,015
cas · a · s	$0{,}1 \cdot 0{,}2 \cdot 0{,}1$	0,002
c · as · as	$0{,}05 \cdot 0{,}15 \cdot 0{,}15$	0,001125
c · a · s · as y c · as · a · s	$0{,}05 \cdot 0{,}2 \cdot 0{,}1 \cdot 0{,}15$	$1{,}5 \cdot 10^{-4}$ cada una
c · a · s · a · s	$0{,}05 \cdot 0{,}2^2 \cdot 0{,}1^2$	$2 \cdot 10^{-5}$

3.2 Segmentación óptima: el algoritmo de Viterbi

El número de segmentaciones crece exponencialmente con la longitud: una cadena de n caracteres tiene hasta 2^{n-1} (ejercicio 3.7). La enumeración del cuadro 3.1 no es viable para textos reales, pero el problema tiene subestructura óptima.

Teorema 3.3 (Viterbi para el modelo unigram). *Sea $x = x_1 \dots x_n$ y sea $\alpha(j)$ la probabilidad máxima de una segmentación del prefijo $x_{1:j}$, con $\alpha(0) = 1$ y $\alpha(j) = 0$ si el prefijo no admite segmentación. Entonces, para $j = 1, \dots, n$,*

$$\alpha(j) = \max_{\substack{0 \le i < j \\ x_{i+1:j} \in \mathcal{V}}} \alpha(i)\, p(x_{i+1:j}),$$

y $\alpha(n) = \max_{s \in S(x)} P(s)$. Si ℓ es la longitud máxima de un token, el cálculo requiere $O(n\ell)$ productos.

Demostración. Sea $s = (t_1, \dots, t_m)$ una segmentación de $x_{1:j}$ y sea $t_m = x_{i+1:j}$ su último token. Entonces $(t_1, \dots, t_{m-1})$ es una segmentación de $x_{1:i}$ y $P(s) = P(t_1, \dots, t_{m-1})\, p(t_m) \le \alpha(i)\, p(x_{i+1:j})$. Esto prueba que $\alpha(j)$ no supera el máximo de la fórmula. Recíprocamente, para cada i con $x_{i+1:j} \in \mathcal{V}$, si s' es una segmentación de $x_{1:i}$ con $P(s') = \alpha(i)$, añadir el token $x_{i+1:j}$ da una segmentación de $x_{1:j}$ con probabilidad $\alpha(i)\, p(x_{i+1:j})$; luego $\alpha(j)$ alcanza el máximo. Por inducción sobre j se obtiene la fórmula. Para cada j solo hay que considerar los i con $j - i \le \ell$, lo que da $O(\ell)$ productos por posición y $O(n\ell)$ en total. La segmentación óptima se recupera guardando el i que alcanza cada máximo y recorriendo esos punteros desde n hacia atrás. ∎

Ejemplo 3.4. Para «casas» con el vocabulario del ejemplo 3.2:

$$\begin{aligned}
&\alpha(1) = p(\texttt{c}) = 0{,}05, \qquad \alpha(2) = \alpha(1)\, p(\texttt{a}) = 0{,}01,\\
&\alpha(3) = \max\{p(\texttt{cas}),\ \alpha(1)\, p(\texttt{as}),\ \alpha(2)\, p(\texttt{s})\} = \max\{0{,}1;\ 0{,}0075;\ 0{,}001\} = 0{,}1,\\
&\alpha(4) = \max\{p(\texttt{casa}),\ \alpha(3)\, p(\texttt{a})\} = \max\{0{,}2;\ 0{,}02\} = 0{,}2,\\
&\alpha(5) = \max\{\alpha(4)\, p(\texttt{s}),\ \alpha(3)\, p(\texttt{as})\} = \max\{0{,}02;\ 0{,}015\} = 0{,}02,
\end{aligned}$$

que coincide con el cuadro 3.1.

3.3 Entrenamiento por EM y poda

Las probabilidades $p(t)$ no se observan, porque el corpus no viene segmentado. Kudo trata la segmentación como una variable oculta y maximiza la verosimilitud marginal [11, §3.2]

$$\mathcal{L} = \sum_{x \in \mathcal{D}} \ln \sum_{s \in S(x)} P(s),$$

donde $\mathcal{D}$ es el corpus de frases. El algoritmo EM alterna dos pasos: el paso E calcula, con las probabilidades actuales, el número esperado de veces que aparece cada token en las segmentaciones de cada frase, y el paso M reestima $p(t)$ como esos recuentos esperados normalizados. Es la misma idea que el estimador de máxima verosimilitud por frecuencias relativas del módulo 1, con recuentos esperados en lugar de observados.

El vocabulario se obtiene podando [11, §3.2]:

1. se parte de un vocabulario semilla grande: todos los caracteres y las subcadenas frecuentes del corpus;
2. con el vocabulario fijo, se optimizan las $p(t)$ por EM;
3. para cada token se estima cuánto bajaría $\mathcal{L}$ si se eliminara, y se conserva el $\eta\,\%$ de los tokens con mayor pérdida (por ejemplo, el 80 %);
4. se repiten los pasos 2 y 3 hasta alcanzar el tamaño deseado.

Los caracteres individuales nunca se eliminan, de modo que toda cadena sigue teniendo al menos una segmentación.

Observación 3.5 (Regularización por subpalabras). Como el modelo define una distribución sobre $S(x)$, durante el entrenamiento de un modelo neuronal se puede muestrear una segmentación distinta en cada paso, con probabilidad proporcional a $P(s)^{\alpha}$. Un α pequeño da muestras casi uniformes; uno grande se acerca a la segmentación de Viterbi. Kudo muestra que este ruido hace más robustos los modelos de traducción [11, §3.3].

Cuadro 3.2: Comparación de BPE y del modelo unigram [11, §3.4].

	BPE	Unigram
Construcción	de abajo arriba (fusiona)	de arriba abajo (poda)
Resultado del entrenamiento	lista ordenada de fusiones	probabilidad de cada token
Codificación	aplicar las fusiones en orden	Viterbi: segmentación más probable
Varias segmentaciones	no; es determinista	sí; se pueden muestrear

3.4 SentencePiece

SentencePiece es una biblioteca que entrena tanto BPE como el modelo unigram directamente sobre frases sin pretokenizar [12]. Trata el texto como una secuencia de caracteres Unicode en la que el espacio es un símbolo más, que se escribe con el metasímbolo _ (`U+2581`); así, «Hello world.» puede segmentarse como `[Hello] [_wor] [ld] [.]` [12, §3.1]. Como el espacio forma parte de los tokens, decodificar consiste en concatenar y sustituir _ por un espacio, y la tokenización es *sin pérdida*:

$$\mathrm{dec}(\mathrm{enc}(\mathcal{N}(x))) = \mathcal{N}(x),$$

donde $\mathcal{N}$ es la normalización del texto, por defecto una variante de la forma NFKC de Unicode [12, §3.4]. Al no depender de los espacios para pretokenizar, el método se aplica igual a idiomas que no separan las palabras, como el japonés. En la literatura, «SentencePiece» se usa a veces como sinónimo de tokenización unigram [10, §2.4.3].

3.5 El tamaño del vocabulario

Proposición 3.6. *Un modelo con vocabulario de tamaño K y dimensión d tiene Kd parámetros en la matriz de embeddings de entrada y otros Kd en la matriz de salida que produce los logits, salvo que ambas se compartan. El coste es lineal en K.*

Demostración. La matriz de entrada $E \in \mathbb{R}^{K\times d}$ tiene una fila por token (tema 4); la de salida, $W \in \mathbb{R}^{d\times K}$, una columna por token. Cada una tiene Kd entradas, y duplicar K con d fijo

duplica ese número. ■

Cuadro 3.3: Coste de la matriz de embeddings en modelos con datos públicos.

Modelo	K	d	Kd
GPT-2 pequeño (2019) [19]	50 257	768	38 597 376
Llama 2 7B (2023) [21]	32 000	4096	131 072 000
Llama 3 8B (2024) [14]	128 256	4096	525 336 576

En Llama 3 8B, solo la matriz de entrada tiene unos $5{,}25 \cdot 10^8$ parámetros, que en `bf16` (2 bytes por parámetro) ocupan 1,05 GB. Según la información publicada, Grok-1 usa 131 072 tokens y Gemma, 256 000.

Definición 3.7 (Compresión). La *compresión* de un tokenizador sobre un texto es el cociente entre el número de bytes UTF-8 del texto y el número de tokens que produce, medido en bytes por token.

Por la proposición 2.6, sobre el propio corpus de entrenamiento cada fusión reduce el número de tokens, de modo que la compresión crece con k; el aumento es cada vez menor, porque los pares que se fusionan son cada vez menos frecuentes. El cuadro 3.4 muestra los resultados de la práctica del módulo, con un BPE de bytes entrenado sobre el fragmento de 2536 bytes de *El Quijote*. Estas cifras sobre el texto de entrenamiento son optimistas: en la misma práctica, entrenando con los dos primeros párrafos y 200 fusiones, la compresión es de 2,616 bytes por token en el texto de entrenamiento y de 1,953 en el tercer párrafo, que el tokenizador no vio.

Cuadro 3.4: Compresión de un BPE de bytes sobre su propio corpus de entrenamiento (práctica del módulo).

Fusiones	0	10	25	50	100	200	300
Tokens	2536	2047	1751	1483	1230	982	819
Bytes por token	1,000	1,239	1,448	1,710	2,062	2,582	3,096

3.6 Efectos de la tokenización

Observación 3.8 (Letras). El modelo recibe enteros, no letras. Para contar las «r» de «ferrocarril», segmentada por ejemplo como `ferro·carril`, tendría que conocer la ortografía interna de cada token, que no figura en la entrada y solo se aprende de forma indirecta.

Observación 3.9 (Números). Si los números se segmentan de forma irregular, el mismo dígito ocupa posiciones distintas dentro de los tokens según el número, lo que dificulta la aritmética [10, ejercicio 2.2]. Por eso algunos tokenizadores pretrocean los números en grupos de hasta tres dígitos antes de aplicar BPE.

Observación 3.10 (Idiomas). Un tokenizador entrenado con poco texto de un idioma lo segmenta en muchos tokens. Con el tokenizador de la práctica, entrenado en español, «El perro ladra en la plaza.» (27 bytes) da 15 tokens; «The dog barks in the square.» (28 bytes), 21, y una frase rusa de 22 bytes, 22 tokens: ninguna fusión encaja con los bytes del alfabeto cirílico. Petrov *et al.* midieron, en tokenizadores comerciales, diferencias de longitud de hasta 15

veces para un mismo texto traducido [18]. El mismo mensaje cuesta más y cabe menos en el contexto.

Lecturas recomendadas para este tema

- Kudo [11], §3.2 «Unigram language model», §3.3 «Subword sampling» y §3.4 «BPE vs. Unigram language model».
- Kudo y Richardson [12], §3.1 «Lossless Tokenization» y §3.4 «Customizable character normalization».
- Jurafsky y Martin [10], §2.4.3 «BPE in practice».
- Petrov *et al.* [18], sobre las diferencias de longitud entre idiomas.

Ejercicios del tema 3

Ejercicio 3.1 ⋆ Dé dos razones por las que un vocabulario de palabras completas es mala idea para un LLM multilingüe.

Ejercicio 3.2 ⋆ Un LLM falla al contar cuántas «r» tiene «ferrocarril». Explique el papel de la tokenización y cómo se puede facilitar la tarea.

Ejercicio 3.3 ⋆ Justifique si son verdaderas o falsas: (a) BPE puede devolver varias segmentaciones de la misma palabra; (b) el tokenizador se entrena a la vez que el Transformer; (c) duplicar K duplica los parámetros de la matriz de embeddings.

Ejercicio 3.4 ⋆⋆ Con el vocabulario del ejemplo 3.2, calcule la pérdida $-\ln P(s^*)$ de «casas». ¿Cuántos nats se pierden si se usa la segmentación $(\texttt{cas}, \texttt{as})$?

Ejercicio 3.5 ⋆⋆ Un modelo tiene $K = 32\,000$, $d = 4096$ y no comparte la matriz de entrada con la de salida. ¿Cuántos parámetros suman ambas? ¿Cuánto ocupan en `bf16`?

Ejercicio 3.6 ⋆⋆ Un tokenizador pretrocea los números en grupos de hasta tres dígitos, de izquierda a derecha, antes de aplicar BPE. (a) Segmente 1605, 2026, 123456 y 1234567. (b) Compare las segmentaciones de 999 y de $999 + 1$. (c) Explique por qué la agrupación de izquierda a derecha dificulta la suma y proponga otra agrupación que respete las unidades, decenas y centenas.

Fuente: adaptado de Jurafsky y Martin [10], cap. 2, ejercicio 2.2 (se sustituye el uso de un visualizador por una regla de agrupación explícita).

Ejercicio 3.7 ⋆⋆⋆ (a) Demuestre que una cadena de $n \geq 1$ caracteres admite exactamente 2^{n-1} segmentaciones en subcadenas no vacías. (b) Sea $\mathcal{V}$ un vocabulario que contiene todos los caracteres y cuyos tokens tienen longitud a lo sumo ℓ. Demuestre que el número de segmentaciones con tokens de $\mathcal{V}$ está acotado por el número de composiciones de n con partes en $\{1, \ldots, \ell\}$, y compare con el coste $O(n\ell)$ del teorema 3.3.

Tema 4

Embeddings y word2vec

Un token es un entero sin estructura: el índice de «gato» no está más cerca del de «perro» que del de «avión». Un *embedding* asigna a cada token un vector de $\mathbb{R}^d$, de modo que la geometría pueda reflejar el significado. Este tema define la matriz de embeddings, presenta la hipótesis distribucional y los vectores de coocurrencia, desarrolla el modelo skip-gram de word2vec con softmax y con muestreo negativo, y demuestra que este último factoriza de forma implícita una matriz de información mutua puntual. Se sigue [10, §5.2–5.5], la clase 2 de CS224N [3] y los artículos de Mikolov *et al.* [15, 16] y de Levy y Goldberg [13].

4.1 La matriz de embeddings

Definición 4.1 (Matriz de embeddings). Sea $\mathcal{V}$ un vocabulario de tamaño K con índices $0, \dots, K-1$. Una *matriz de embeddings* es una matriz $E \in \mathbb{R}^{K \times d}$; el *embedding* del token i es su fila $E_{i,:} \in \mathbb{R}^{1 \times d}$.

Proposición 4.2. *Sea $e_i \in \mathbb{R}^K$ el vector* one-hot *del token i, con un 1 en la posición i y ceros en el resto. Entonces $e_i^\top E = E_{i,:}$.*

Demostración. La componente j de $e_i^\top E$ es $\sum_k (e_i)_k E_{kj} = E_{ij}$, porque $(e_i)_k = 0$ para $k \neq i$ y $(e_i)_i = 1$. ∎

La proposición 4.2 muestra que la capa de embeddings es una capa lineal sin sesgo aplicada a una entrada *one-hot*. En la práctica no se multiplica, porque el producto costaría Kd operaciones y la indexación $E[i]$ no cuesta ninguna; pero la equivalencia explica cómo se calcula el gradiente: solo las filas de los tokens presentes reciben gradiente distinto de cero (módulo 9).

Definición 4.3 (Similitud coseno). Para $a, b \in \mathbb{R}^d$ no nulos, la *similitud coseno* es $\cos(a, b) = \dfrac{a \cdot b}{\|a\| \, \|b\|} \in [-1, 1]$.

Por la desigualdad de Cauchy–Schwarz (módulo 2), $|\cos(a, b)| \leq 1$, con igualdad si y solo si los vectores son proporcionales. El coseno no cambia al multiplicar un vector por una constante positiva (ejercicio 4.3), y por eso no penaliza que una palabra sea más frecuente que otra [10, §5.4].

4.2 La hipótesis distribucional

La *hipótesis distribucional* afirma que las palabras que aparecen en contextos parecidos tienen significados parecidos; Se atribuye a Firth la formulación según la cual una palabra se conoce por las palabras que la acompañan, que la tarea 2 de CS224N toma como punto de partida de

word2vec [3]; véase también [10, §5.2]. Si «gato» y «perro» aparecen junto a «come», «duerme» y «pelo», sus representaciones deben parecerse.

Definición 4.4 (Matriz de coocurrencias). Fijada una ventana de m palabras, la *matriz de coocurrencias* $N \in \mathbb{R}^{K\times K}$ tiene por entrada $N(w,c)$ el número de veces que la palabra c aparece a distancia a lo sumo m de w en el corpus. La fila $N(w,:)$ es una representación de w por recuentos [10, §5.3].

Los recuentos están dominados por las palabras frecuentes («de», «que»), que aparecen junto a todas. La información mutua puntual del módulo 6 corrige ese efecto comparando la frecuencia observada de un par con la esperada si las palabras fueran independientes.

Definición 4.5 (Información mutua puntual). Sean $P(w,c) = N(w,c)/|\mathcal{D}|$, $P(w) = \sum_c P(w,c)$ y $P(c) = \sum_w P(w,c)$, con $|\mathcal{D}| = \sum_{w,c} N(w,c)$ el número de pares. La *información mutua puntual* y su versión positiva son

$$\mathrm{PMI}(w,c) = \ln \frac{P(w,c)}{P(w)\,P(c)}, \qquad \mathrm{PPMI}(w,c) = \max\{\mathrm{PMI}(w,c), 0\}.$$

La PMI vale 0 si w y c son independientes, es positiva si coinciden más de lo esperado y vale $-\infty$ si el par no se observa nunca; la PPMI evita ese valor. La información mutua del módulo 6 es la esperanza de la PMI bajo $P(w,c)$.

4.3 Skip-gram

Los vectores de recuentos tienen dimensión K y casi todas sus entradas son nulas. Word2vec aprende directamente vectores densos de dimensión $d \ll K$ con una tarea de predicción [15].

Definición 4.6 (Pares de skip-gram). Sea una frase $w_1, \dots, w_n$ y una ventana $m \geq 1$. Los *pares de entrenamiento* de skip-gram son los pares ordenados (centro, contexto) (w_t, w_{t+j}) con $1 \leq |j| \leq m$ y $1 \leq t+j \leq n$.

Proposición 4.7. *Si $n \geq m+1$, una frase de n palabras da $m(2n-m-1)$ pares con ventana m.*

Demostración. Para cada distancia $j \in \{1,\dots,m\}$ hay $n-j$ pares (w_t, w_{t+j}) hacia la derecha y otros $n-j$ hacia la izquierda. El total es $2\sum_{j=1}^{m}(n-j) = 2\big(nm - \frac{m(m+1)}{2}\big) = m(2n-m-1)$. ∎

Por ejemplo, «el gato come pescado» ($n=4$) da $1\cdot(8-1-1) = 6$ pares con $m=1$, (el, gato), (gato, el), (gato, come), (come, gato), (come, pescado) y (pescado, come), y $2\cdot(8-2-1) = 10$ con $m=2$.

Definición 4.8 (Skip-gram con softmax). Cada palabra w tiene dos vectores: $v_w \in \mathbb{R}^d$ cuando es centro y $u_w \in \mathbb{R}^d$ cuando es contexto. El modelo es

$$P(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_{w\in\mathcal{V}} \exp(u_w^\top v_c)} = \mathrm{softmax}(U^\top v_c)_o,$$

donde $U \in \mathbb{R}^{d\times K}$ tiene por columnas los u_w. La pérdida de un par es $J = -\ln P(o \mid c)$, la entropía cruzada entre el vector *one-hot* y de o y la predicción $\hat{y} = \mathrm{softmax}(U^\top v_c)$ [3].

Proposición 4.9 (Gradiente respecto al vector central).

$$\frac{\partial J}{\partial v_c} = \sum_{w \in \mathcal{V}} P(w \mid c)\, u_w - u_o.$$

Demostración. $J = -u_o^\top v_c + \ln \sum_w \exp(u_w^\top v_c)$. El gradiente del primer término es $-u_o$. Para el segundo, por la regla de la cadena, $\nabla_{v_c} \ln \sum_w \exp(u_w^\top v_c) = \dfrac{\sum_w \exp(u_w^\top v_c)\, u_w}{\sum_{w'} \exp(u_{w'}^\top v_c)} = \sum_w P(w \mid c)\, u_w$. ∎

El gradiente es «lo esperado menos lo observado»: el descenso de gradiente acerca v_c al vector del contexto real y lo aleja de la media de los vectores de contexto que el modelo predice. Calcular esa media cuesta $O(Kd)$ por par, demasiado para $K \sim 10^5$ y miles de millones de pares.

4.4 Muestreo negativo

Definición 4.10 (Skip-gram con muestreo negativo). Sean w la palabra central, c_+ un contexto real y $c_1, \ldots, c_k$ palabras de ruido extraídas al azar. La pérdida de *skip-gram con muestreo negativo* (SGNS) es

$$L = -\ln \sigma(c_+ \cdot w) - \sum_{i=1}^{k} \ln \sigma(-c_i \cdot w),$$

donde w denota el vector central y c_+, c_i los vectores de contexto [10, §5.5.2]. Mikolov *et al.* extraen el ruido de la distribución unigram elevada a $3/4$ y renormalizada [16].

La softmax sobre K palabras se sustituye por $k+1$ regresiones logísticas (módulo 8): distinguir el par real de k pares falsos. El coste por par pasa a $O(kd)$, con k entre 2 y 20.

Proposición 4.11 (Gradientes de SGNS).

$$\frac{\partial L}{\partial c_+} = \big(\sigma(c_+ \cdot w) - 1\big) w, \qquad \frac{\partial L}{\partial c_i} = \sigma(c_i \cdot w)\, w, \qquad \frac{\partial L}{\partial w} = \big(\sigma(c_+ \cdot w) - 1\big) c_+ + \sum_{i=1}^{k} \sigma(c_i \cdot w)\, c_i.$$

La demostración es el ejercicio 4.6. Los signos confirman la intuición: el paso de descenso aumenta $c_+ \cdot w$ y disminuye cada $c_i \cdot w$.

Ejemplo 4.12. Sean $w = v_{\text{gato}} = (1, 0)$, $c_+ = u_{\text{come}} = (0, 1)$ y un único negativo $c_1 = u_{\text{avión}} = (-1, 0)$. Los productos son $c_+ \cdot w = 0$ y $c_1 \cdot w = -1$, luego $\sigma(0) = 0{,}5$ y $\sigma(-c_1 \cdot w) = \sigma(1) = 0{,}7311$. La pérdida es $L = -\ln 0{,}5 - \ln 0{,}7311 = 0{,}6931 + 0{,}3133 = 1{,}006$.

4.5 Muestreo negativo y PMI

Teorema 4.13 (Levy y Goldberg). *Sea $N(w, c)$ el número de veces que el par (w, c) aparece en el corpus, $N(w)$ y $N(c)$ sus marginales y $|\mathcal{D}|$ el número total de pares, y supóngase que los k negativos de cada par se extraen de la distribución unigram de contextos $P(c) = N(c)/|\mathcal{D}|$.*

Si la dimensión es suficiente para que cada producto $x = u_c \cdot v_w$ pueda elegirse libremente, la pérdida total esperada de SGNS se minimiza en

$$u_c \cdot v_w = \mathrm{PMI}(w, c) - \ln k$$

para todo par con $N(w, c) > 0$.

Demostración. La pérdida total esperada es la suma, sobre todos los pares, de los términos positivos más k veces la esperanza del término negativo. Agrupando por (w, c), el término que depende de $x = u_c \cdot v_w$ es

$$\ell(x) = -N(w, c) \ln \sigma(x) - k\, N(w)\, P(c) \ln \sigma(-x),$$

porque w aparece como centro $N(w)$ veces y en cada una se espera extraer c como negativo $k\, P(c)$ veces. Con $\frac{d}{dx} \ln \sigma(x) = \sigma(-x)$ y $\frac{d}{dx} \ln \sigma(-x) = -\sigma(x)$,

$$\ell'(x) = -N(w, c)\, \sigma(-x) + k\, N(w) P(c)\, \sigma(x).$$

Igualando a cero y usando $\sigma(x)/\sigma(-x) = e^x$, se obtiene $e^x = \dfrac{N(w, c)}{k\, N(w) P(c)} = \dfrac{1}{k} \cdot \dfrac{N(w, c)\, |\mathcal{D}|}{N(w)\, N(c)}$, es decir, $x = \mathrm{PMI}(w, c) - \ln k$. Como $\ell''(x) = \big(N(w, c) + kN(w)P(c)\big)\sigma(x)\sigma(-x) > 0$, ℓ es estrictamente convexa y el punto crítico es su mínimo [13]. ■

Observación 4.14. El teorema 4.13 une los dos enfoques de este tema: el modelo predictivo de word2vec aprende, en el óptimo, la misma información que la matriz de PMI de los métodos de recuentos, comprimida en vectores de dimensión d. Es la información mutua del módulo 6 convertida en geometría.

4.6 Embeddings en un LLM

En un LLM, la matriz E se aprende junto con el resto del modelo, sin entrenamiento previo con word2vec, y a menudo se comparte con la matriz de salida (*weight tying*) [10, §14.2.3]. La intuición distribucional se mantiene: tras el entrenamiento, tokens con usos parecidos tienen embeddings cercanos. Pero un embedding de esta tabla es *estático*: «banco» tiene el mismo vector en «banco de peces» y en «Banco de España». En el módulo 13 la atención mezcla los vectores según el contexto y produce representaciones contextuales.

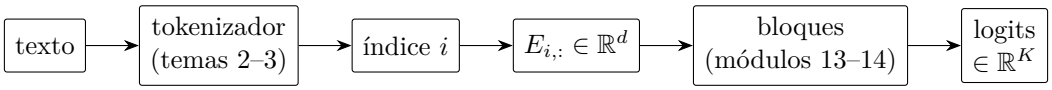


Figura 4.1: Del texto a los logits en un LLM. El tokenizador es fijo; la matriz E y los bloques se entrenan juntos.

Lecturas recomendadas para este tema

- Jurafsky y Martin [10], §5.2 «Vector Semantics: The Intuition», §5.3 «Simple count-based embeddings», §5.4 «Cosine for Measuring Similarity» y §5.5 «Word2vec».
- Stanford CS224N [3], clase 2 «Word Vectors» y tarea 2, pregunta 1.
- Mikolov *et al.* [15, 16]: skip-gram, muestreo negativo y submuestreo de palabras frecuentes.
- Levy y Goldberg [13]: SGNS como factorización implícita de la matriz de PMI desplazada.

Ejercicios del tema 4

Ejercicio 4.1 ⋆ Sea $E = \begin{pmatrix} 1 & 0 \\ 2 & -1 \\ 0 & 3 \end{pmatrix}$. Calcule $e_1^\top E$ (con índices $0, 1, 2$) haciendo el producto completo y compruebe la proposición 4.2. ¿Cuántas multiplicaciones requeriría el producto si $K = 50\,257$ y $d = 768$?

Ejercicio 4.2 ⋆⋆ Liste los pares (centro, contexto) de skip-gram con ventana 1 para «la luna brilla». ¿Cuántos salen con ventana 2? Compruebe ambos resultados con la proposición 4.7.

Ejercicio 4.3 ⋆⋆ Cada palabra se representa por el número de veces que aparece junto a «tarta» y junto a «resultado»: cereza $= (40, 3)$, fresa $= (20, 4)$, digital $= (2, 30)$. (a) Calcule $\cos(\text{cereza}, \text{fresa})$ y $\cos(\text{cereza}, \text{digital})$. ¿Qué par es más parecido? (b) Demuestre que $\cos(\lambda a, b) = \cos(a, b)$ para todo $\lambda > 0$ y explique por qué esta propiedad es deseable.

Fuente: adaptado de Jurafsky y Martin [10], cap. 5, ejercicios 5.1 y 5.2 (palabras traducidas y recuentos cambiados).

Ejercicio 4.4 ⋆⋆ Un corpus da los siguientes recuentos de coocurrencia: «gato» con «come» 4 veces y con «maúlla» 4 veces; «perro» con «come» 4 veces y con «ladra» 4 veces. (a) Calcule PMI y PPMI de los seis pares (gato o perro con come, maúlla o ladra). (b) Según el teorema 4.13, ¿qué producto $u_c \cdot v_w$ aprendería SGNS con $k = 2$ para los pares observados?

Ejercicio 4.5 ⋆⋆⋆ En el modelo de la definición 4.8: (a) escriba $\partial J / \partial v_c$ en forma vectorial en función de U, y e $\hat{y}$, y diga cuándo se anula; (b) calcule $\partial J / \partial u_w$ para $w = o$ y para $w \neq o$; (c) evalúe los gradientes con $\mathcal{V} = \{\text{come}, \text{avión}\}$, $u_{\text{come}} = (0, 1)$, $u_{\text{avión}} = (-1, 0)$, $v_c = v_{\text{gato}} = (1, 0)$ y $o = \text{come}$.

Fuente: adaptado de Stanford CS224N [3], tarea 2, pregunta 1, apartados (b) y (e) (se añade el apartado numérico c).

Ejercicio 4.6 ⋆⋆⋆ Con un solo negativo, $L = -\ln \sigma(c_+ \cdot w) - \ln \sigma(-c_- \cdot w)$. (a) Deduzca $\partial L / \partial c_+$, $\partial L / \partial c_-$ y $\partial L / \partial w$, lo que demuestra la proposición 4.11 con $k = 1$. (b) Evalúelos en el ejemplo 4.12. (c) Dé un paso de descenso de gradiente sobre w con $\eta = 0{,}1$ y compruebe que la pérdida baja.

Fuente: adaptado de Jurafsky y Martin [10], cap. 5, ejercicio 5.3 (se añaden los apartados numéricos b y c).

Tema 5

Modelos secuenciales: n-gramas, RNN y LSTM

Con tokens y embeddings ya definidos, queda modelar la dependencia entre posiciones. Este tema repasa los tres modelos que precedieron al Transformer y demuestra por qué ninguno escala: los n-gramas no generalizan, las redes recurrentes sufren el desvanecimiento del gradiente y todas las arquitecturas recurrentes son secuenciales en el tiempo. Se sigue [10, cap. 14], la clase 4 de CS224N [3], [8, cap. 10] y [23, §9.7 y §10.1]; los resultados sobre el gradiente proceden de Bengio, Simard y Frasconi [2] y de Pascanu, Mikolov y Bengio [17, §2–3], y la LSTM, de Hochreiter y Schmidhuber [9].

5.1 N-gramas y dispersión

Un modelo de n-gramas (módulo 1) estima $P(x_t \mid x_{t-n+1:t-1})$ por frecuencias relativas y necesita una tabla de K^n entradas.

Proposición 5.1. *En un corpus de $N \geq n$ tokens aparecen a lo sumo $N - n + 1$ n-gramas distintos. Por tanto, la fracción de casillas de la tabla con recuento positivo es a lo sumo $(N - n + 1)/K^n$.*

Demostración. Un corpus de N tokens tiene $N - n + 1$ posiciones de comienzo para un n-grama, y cada posición aporta un solo n-grama. El número de n-gramas distintos no supera el de posiciones. ∎

Ejemplo 5.2. El fragmento de *El Quijote* de la práctica tiene 440 palabras y 243 distintas. Hay $243^2 = 59\,049$ bigramas posibles, pero solo aparecen 416 distintos, el 0,70 %, y 398 de ellos una sola vez. Con un vocabulario de $5 \cdot 10^4$ tokens, los trigramas posibles son $1{,}25 \cdot 10^{14}$, muchos más que los tokens de cualquier corpus.

El defecto de fondo no es solo el tamaño: cada contexto es una casilla aislada y el modelo no sabe que «el gato come» y «el perro come» se parecen. El modelo neuronal de Bengio *et al.* [1], estudiado en el módulo 9, resuelve la generalización con embeddings, pero conserva un contexto de longitud fija.

5.2 Redes recurrentes

Definición 5.3 (Red recurrente). Sean $x_1, \ldots, x_T \in \mathbb{R}^d$ los embeddings de una secuencia. Una *red recurrente* (RNN) simple con estado de dimensión h calcula

$$h_t = \tanh(W h_{t-1} + U x_t + b), \qquad z_t = W_{\mathrm{s}}\, h_t + b_{\mathrm{s}},$$

con $h_0 = 0$, $W \in \mathbb{R}^{h\times h}$, $U \in \mathbb{R}^{h\times d}$, $b \in \mathbb{R}^h$ y una capa de salida $W_{\mathrm{s}} \in \mathbb{R}^{K\times h}$ que produce los logits z_t del token siguiente [10, §14.1–14.2].

Los mismos parámetros se usan en todas las posiciones. El estado h_t resume todo el prefijo $x_{1:t}$, de modo que el contexto no tiene, en principio, longitud fija.

Proposición 5.4. *La parte recurrente de una RNN simple tiene $h(h+d)+h$ parámetros, independientemente de T.*

Demostración. W tiene h^2 entradas, U tiene hd y b tiene h; la suma es $h^2+hd+h = h(h+d)+h$. Ninguna depende de la longitud de la secuencia porque los parámetros se comparten entre posiciones. ∎

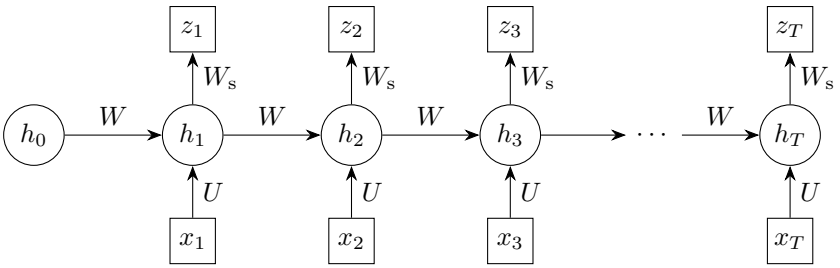


Figura 5.1: RNN desplegada en el tiempo. Los arcos horizontales forman una cadena de T dependencias: para calcular h_T hay que calcular antes $h_1, \dots, h_{T-1}$, y el gradiente recorre la misma cadena en sentido inverso.

5.3 Retropropagación en el tiempo y desvanecimiento del gradiente

El gradiente de una RNN se calcula aplicando la retropropagación al grafo desplegado de la figura 5.1; el procedimiento se llama *retropropagación en el tiempo* [23, §9.7].

Lema 5.5. *Sea $a_t = Wh_{t-1} + Ux_t + b$. El jacobiano de un paso es $\dfrac{\partial h_t}{\partial h_{t-1}} = D_t W$, con $D_t = \operatorname{diag}\big(1 - \tanh^2(a_t)\big)$, y*

$$\frac{\partial h_T}{\partial h_0} = D_T W \, D_{T-1} W \cdots D_1 W.$$

Demostración. Por la regla de la cadena (módulo 4), $\partial h_t/\partial h_{t-1} = \tanh'(a_t)\cdot \partial a_t/\partial h_{t-1}$, donde $\tanh'$ se aplica componente a componente y forma la matriz diagonal D_t, ya que $\tanh'(u) = 1-\tanh^2(u)$, y $\partial a_t/\partial h_{t-1} = W$. El jacobiano de una composición es el producto de los jacobianos. ∎

Teorema 5.6 (Cota exponencial del gradiente). *Sea $\|\cdot\|$ la norma espectral. En la RNN de la definición 5.3,*

$$\left\| \frac{\partial h_T}{\partial h_0} \right\| \le \|W\|^T.$$

En particular, si $\|W\| \le \gamma < 1$, el gradiente que llega a h_0 desde la pérdida en la posición T decae al menos como γ^T.

Demostración. Las entradas de D_t son $1-\tanh^2(\cdot) \in (0,1]$, y la norma espectral de una matriz diagonal es el máximo valor absoluto de su diagonal, luego $\|D_t\| \le 1$. La norma espectral es submultiplicativa (módulo 3), de modo que, por el lema 5.5, $\|\partial h_T/\partial h_0\| \le \prod_{t=1}^{T}\|D_t\|\,\|W\| \le \|W\|^T$. Finalmente, si L depende de h_T, $\partial L/\partial h_0 = (\partial h_T/\partial h_0)^\top \partial L/\partial h_T$ y $\|\partial L/\partial h_0\| \le \gamma^T \|\partial L/\partial h_T\|$. ∎

Corolario 5.7. *Para que el gradiente pueda crecer exponencialmente (*explotar*) es necesario que $\|W\| > 1$. Con una activación de derivada acotada por γ_ϕ (por ejemplo, $\gamma_\phi = 1/4$ para la sigmoide), la condición necesaria es $\|W\| > 1/\gamma_\phi$ [17, §2.1].*

Demostración. Con una activación ϕ cualquiera, $\|D_t\| \le \gamma_\phi$, y el argumento del teorema 5.6 da $\|\partial h_T/\partial h_0\| \le (\gamma_\phi \|W\|)^T$. Si $\gamma_\phi\|W\| \le 1$, la cota no crece con T; por contraposición, el crecimiento exponencial exige $\gamma_\phi\|W\| > 1$. ∎

Cuadro 5.1: Factor w^T en la RNN escalar lineal $h_t = w\,h_{t-1}$.

w	$T=10$	$T=50$	$T=100$
0,9	0,3487	$5{,}154\cdot10^{-3}$	$2{,}656\cdot10^{-5}$
0,99	0,9044	0,6050	0,3660
1,1	2,594	117,4	13 781

El caso escalar lineal del cuadro 5.1 muestra el fenómeno: con $w = 0{,}9$, la señal de un token situado 100 posiciones atrás llega multiplicada por $2{,}7\cdot10^{-5}$, y la red no aprende dependencias largas. Bengio, Simard y Frasconi demostraron que, para que una red recurrente almacene información de forma robusta frente al ruido, debe operar en una región en la que el gradiente se desvanece, de modo que aprender dependencias largas por descenso de gradiente es intrínsecamente difícil [2].

Definición 5.8 (Recorte del gradiente). Dado un umbral $\tau > 0$, el *recorte por norma* sustituye el gradiente g por

$$\operatorname{clip}_\tau(g) = \begin{cases} g & \text{si } \|g\| < \tau, \\ \tau\, g/\|g\| & \text{si } \|g\| \ge \tau. \end{cases}$$

Pascanu *et al.* proponen esta regla contra la explosión del gradiente y sugieren un umbral entre la mitad y diez veces la norma media del gradiente [17, §3.2].

Proposición 5.9. *$\|\operatorname{clip}_\tau(g)\| \le \tau$, y $\operatorname{clip}_\tau(g)$ es un múltiplo positivo de g; en particular, $g \cdot \operatorname{clip}_\tau(g) \ge 0$ y el paso recortado sigue siendo de descenso.*

Demostración. Si $\|g\| < \tau$, la conclusión es inmediata. Si no, $\operatorname{clip}_\tau(g) = \lambda g$ con $\lambda = \tau/\|g\| \in (0,1]$, luego $\|\operatorname{clip}_\tau(g)\| = \lambda\|g\| = \tau$ y $g\cdot\operatorname{clip}_\tau(g) = \lambda\|g\|^2 \ge 0$. ∎

El recorte resuelve la explosión, pero no el desvanecimiento: un gradiente pequeño no se agranda.

5.4 La LSTM

Definición 5.10 (LSTM). Una LSTM añade al estado oculto h_t una *celda de memoria* $c_t \in \mathbb{R}^h$ y tres *puertas* con valores en $(0,1)^h$:

$$\begin{aligned} f_t &= \sigma(W_f h_{t-1} + U_f x_t + b_f), & i_t &= \sigma(W_i h_{t-1} + U_i x_t + b_i), \\ o_t &= \sigma(W_o h_{t-1} + U_o x_t + b_o), & \tilde{c}_t &= \tanh(W_c h_{t-1} + U_c x_t + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

La puerta de *olvido* f_t decide qué parte de la memoria se conserva; la de *entrada* i_t, qué parte del candidato $\tilde{c}_t$ se añade, y la de *salida* o_t, qué parte de la memoria pasa al estado oculto [9], [10, §14.5], [23, §10.1].

La puerta de olvido no figuraba en el artículo original de 1997; la introdujeron Gers, Schmidhuber y Cummins [7], y es la forma que usan los libros de texto actuales.

Proposición 5.11. *El jacobiano de c_t respecto a c_{t-1} a través del camino directo, es decir, manteniendo fijas las puertas y el candidato, es* $\operatorname{diag}(f_t)$*. A lo largo de T pasos, el producto de estos jacobianos es* $\operatorname{diag}\big(\prod_{t=1}^{T} f_t\big)$*, con el producto componente a componente.*

Demostración. Con f_t, i_t y $\tilde{c}_t$ fijos, c_t es una función afín de c_{t-1} cuya parte lineal es $c_{t-1} \mapsto f_t \odot c_{t-1}$, de jacobiano $\operatorname{diag}(f_t)$. Un producto de matrices diagonales es la matriz diagonal del producto de las diagonales. ∎

La celda se actualiza *sumando* y no multiplicando por una matriz W: si la red aprende $f_t \approx 1$, el gradiente recorre muchos pasos casi sin atenuarse. Con $f_t = 0{,}98$ constante, el factor a 100 pasos es $0{,}98^{100} = 0{,}1326$, frente a $2{,}7 \cdot 10^{-5}$ en la RNN con $w = 0{,}9$. Es la misma idea que la conexión residual del módulo 11. En la práctica, las LSTM aprovechan contextos de cientos de tokens.

Corolario 5.12. *Una LSTM con estado de dimensión h y entrada de dimensión d tiene $4\big(h(h+d)+h\big)$ parámetros recurrentes, cuatro veces los de una RNN simple.*

Demostración. Cada una de las cuatro transformaciones (f_t, i_t, o_t y $\tilde{c}_t$) tiene una matriz $h \times h$, una $h \times d$ y un sesgo de dimensión h, como la RNN de la proposición 5.4. ∎

5.5 Secuencialidad y el paso al Transformer

El segundo límite no tiene que ver con el gradiente. En una red recurrente, h_t depende de h_{t-1}, de modo que calcular h_T exige T pasos que no pueden ejecutarse a la vez dentro de una secuencia (figura 5.1). Una GPU tiene miles de núcleos que quedan ociosos durante esa cadena. El Transformer del módulo 13 calcula las representaciones de todas las posiciones de una capa a la vez, con productos de matrices, y conecta cualquier par de posiciones en un solo paso de atención, a cambio de un coste $O(n^2)$ en la longitud.

Lecturas recomendadas para este tema

– Jurafsky y Martin [10], cap. 14 «RNNs and LSTMs»: §14.1–14.2 (RNN y modelos de lenguaje recurrentes) y §14.5 «The LSTM».

Cuadro 5.2: Modelos de secuencias y sus limitaciones.

Modelo	Contexto	Paralelo	Problema principal
N-grama	$n-1$ fijo	sí (contar)	tabla de K^n casillas; no generaliza
MLP (módulo 9)	fijo	sí	contexto corto y fijo
RNN	ilimitado en teoría	no	desvanecimiento del gradiente; secuencial
LSTM	cientos de tokens	no	secuencial; aprovecha mal la GPU
Transformer	ventana de contexto	sí	atención $O(n^2)$ (módulo 13)

- Stanford CS224N [3], clase 4 «Language Models and RNNs».
- Goodfellow, Bengio y Courville [8], cap. 10 «Sequence Modeling: Recurrent and Recursive Nets», en particular las secciones sobre dependencias largas y LSTM.
- Zhang *et al.* [23], §9.7 «Backpropagation Through Time» y §10.1 «Long Short-Term Memory (LSTM)».
- Pascanu, Mikolov y Bengio [17], §2.1 «The mechanics» y §3.2 «Scaling down the gradients».

Ejercicios del tema 5

Ejercicio 5.1 ⋆ El fragmento de *El Quijote* tiene 440 palabras y 243 distintas, y en él aparecen 416 bigramas distintos, 398 de ellos una sola vez. (a) Compruebe la proposición 5.1 con $n = 2$. (b) ¿Qué porcentaje de los bigramas observados es único? (c) ¿Qué estimaría el EMV para un bigrama no observado y qué consecuencia tiene sobre la perplejidad?

Ejercicio 5.2 ⋆⋆ Un gradiente vale $g = (3, -4, 12)$ y el umbral de recorte es $\tau = 5$. Calcule $\mathrm{clip}_\tau(g)$, compruebe su norma y verifique que tiene la misma dirección que g.

Ejercicio 5.3 ⋆⋆ En una LSTM, supóngase que la puerta de olvido vale f en todas las componentes y en todos los pasos. (a) Calcule el factor del camino directo a 100 pasos para $f = 0{,}98$ y compárelo con el de una RNN escalar con $w = 0{,}9$. (b) ¿Qué valor mínimo de f garantiza que el factor sea al menos 0,5 a 100 pasos?

Ejercicio 5.4 ⋆⋆ Para un estado de dimensión $h = 256$ y una entrada de dimensión $d = 256$, compare el número de parámetros recurrentes de una RNN simple, de una LSTM y de una GRU (que tiene tres transformaciones del mismo tamaño). Si el coste de un paso es proporcional al número de parámetros, ¿cuántas veces más cuesta un paso de LSTM que uno de RNN?

Fuente: adaptado de Zhang *et al.* [23], §10.1, ejercicio 3 (se fijan las dimensiones y se pide el cálculo explícito).

Ejercicio 5.5 ⋆⋆⋆ En la RNN escalar lineal $h_t = w\, h_{t-1}$ con $0 < w < 1$, ¿qué valor mínimo de w garantiza que el factor del gradiente a 100 pasos sea al menos 0,1? Interprete el resultado.

Ejercicio 5.6 ⋆⋆⋆ Sea $M \in \mathbb{R}^{n\times n}$ simétrica, con valores propios $\lambda_1, \dots, \lambda_n$ y vectores propios ortonormales $v_1, \dots, v_n$, ordenados de modo que $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|$. (a) Demuestre que M^k tiene valores propios λ_i^k. (b) Sea $x = \sum_i a_i v_i$ con $a_1 \neq 0$. Demuestre que la tangente del

ángulo entre $M^k x$ y v_1 es a lo sumo $(|\lambda_2|/|\lambda_1|)^k \, \|x - a_1 v_1\|/|a_1|$. (c) Explique qué implica para el gradiente de una RNN lineal $h_t = M h_{t-1}$.

Fuente: adaptado de Zhang *et al.* [23], §9.7, ejercicio 1 (se precisa la cota del apartado b).

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa; los resultados exactos se dan como fracción cuando existe.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. un decodificador que aplica la decodificación UTF-8 a cada byte por separado.

Planteamiento. un carácter no ASCII ocupa varios bytes (cuadro 1.2) y ninguno de ellos es por sí solo una palabra del código.

Desarrollo.

1. (a) «año» se codifica como $(97, 195, 177, 111) = (\texttt{61}, \texttt{C3}, \texttt{B1}, \texttt{6F})$. El byte 97 da «a», pero $\texttt{C3} = \texttt{11000011}$ es un byte inicial que anuncia un carácter de 2 bytes; aislado, la cadena está incompleta y la decodificación falla.
2. El error de fondo es suponer que un byte es un carácter, lo que solo es cierto en ASCII. Con «hola» el programa funciona, y por eso el fallo pasa inadvertido.
3. (b) La cadena `C3 28`: el primer byte exige un segundo byte que empiece por `10`, pero $\texttt{28} = \texttt{00101000}$ empieza por `00`. Otra respuesta válida es `FF FF`, porque ningún byte UTF-8 empieza por `11111`.

Resultado. **(a) «año» falla porque la «ñ» = `C3 B1` se parte y ninguna mitad es un carácter; (b) `C3 28` (o `FF FF`) no es UTF-8 válido.**

Comprobación. en Python, `b"\xc3\x28".decode()` lanza el error *invalid continuation byte* y `b"\xff\xff".decode()`, *invalid start byte*.

Solución del ejercicio 1.2

Datos. «é» = `U+00E9`.

Planteamiento. $\texttt{E9} > \texttt{7F}$, luego se usa la fila de 2 bytes, `110xxxxx 10xxxxxx`, con 11 bits útiles.

Desarrollo.

1. Hexadecimal a binario: $\texttt{E} = \texttt{1110}$ y $\texttt{9} = \texttt{1001}$, luego $\texttt{E9} = \texttt{11101001} = 233$.
2. Con 11 bits: `00011101001`; en grupos de $5 + 6$: `00011` y `101001`.
3. Primer byte: $\texttt{110}\,\texttt{00011} = \texttt{11000011} = \texttt{C3} = 195$.
4. Segundo byte: $\texttt{10}\,\texttt{101001} = \texttt{10101001} = \texttt{A9} = 169$.

Resultado. `11000011 10101001` = `C3 A9`.

Comprobación. al quitar las cabeceras `110` y `10` queda `00011 101001` = `E9`. En Python, `"é".encode()` da `b"\xc3\xa9"`.

Solución del ejercicio 1.3

Datos. «pingüino», «Ñandú» y «café» seguido de espacio y de `U+2615`.

Planteamiento. sumar los bytes de cada carácter según su fila en el cuadro 1.2.

Desarrollo.

1. «pingüino» tiene 8 caracteres: siete ASCII (p, i, n, g, i, n, o), que suman $7 \cdot 1 = 7$ bytes, y «ü» (`U+00FC`), con 2. Total: $7 + 2 = 9$.
2. «Ñandú» tiene 5 caracteres: «Ñ» (`U+00D1`) con 2, «a», «n», «d» con $3 \cdot 1 = 3$ y «ú» (`U+00FA`) con 2. Total: $2 + 3 + 2 = 7$.
3. El tercer texto tiene 6 caracteres: «c», «a», «f» (3 bytes), «é» (2), el espacio (1) y `U+2615`, que está en el intervalo `U+0800`–`U+FFFF` y ocupa 3. Total: $3 + 2 + 1 + 3 = 9$.

Resultado. **9, 7 y 9 bytes.**

Comprobación. bytes menos caracteres: $9 - 8 = 1$ («ü»), $7 - 5 = 2$ («Ñ» y «ú») y $9 - 6 = 3$ («é» aporta 1 y `U+2615` aporta 2).

Solución del ejercicio 1.4

Datos. «¿» = `U+00BF`; carácter chino `U+8FDB`.

Planteamiento. `BF` está en la fila de 2 bytes (11 bits útiles); `8FDB` está en la de 3 bytes (16 bits útiles, patrón `1110xxxx 10xxxxxx 10xxxxxx`).

Desarrollo.

1. «¿»: `BF` = `10111111` = 191. Con 11 bits, `00010111111`; en grupos de $5 + 6$, `00010` y `111111`.
2. Bytes: `110 00010` = `11000010` = `C2` y `10 111111` = `10111111` = `BF`.
3. `U+8FDB`: `8FDB` = `1000 1111 1101 1011` (16 bits); en grupos de $4 + 6 + 6$, `1000`, `111111` y `011011`.
4. Bytes: `1110 1000` = `E8`, `10 111111` = `BF` y `10 011011` = `10011011` = `9B`.

Resultado. **«¿»** = `C2 BF` **(2 bytes);** `U+8FDB` = `E8 BF 9B` **(3 bytes).**

Comprobación. al quitar las cabeceras del segundo carácter queda `1000 111111 011011` = `8FDB`. Obsérvese que «¿» empieza por `C2` y no por `C3`, como las vocales con tilde, porque `BF` < `C0`.

Solución del ejercicio 1.5

Datos. el cuadro 1.2; emoji `U+1F600`.

Planteamiento. cardinal de cada intervalo y cociente entre la fila de 4 bytes y el total de valores escalares.

Desarrollo.

1. 1 byte: `80` = 128. 2 bytes: `800` − `80` = 2048 − 128 = 1920.

2. 3 bytes: `10000` − `800` = 65 536 − 2048 = 63 488, de los que 2048 son sustitutos.
3. 4 bytes: `110000` − `10000` = 1 114 112 − 65 536 = 1 048 576 = 2^{20}.
4. Valores escalares: 128 + 1920 + (63 488 − 2048) + 1 048 576 = 1 112 064.
5. Fracción con 4 bytes: 1 048 576/1 112 064 = 0,9429.
6. `1F600` ≥ `10000`, luego está en la última fila. Sus bytes son `F0 9F 98 80`.

Resultado. **el** 94,29 % **de los valores escalares necesita 4 bytes**; el emoji ocupa 4 bytes porque su punto de código supera `FFFF`.

Comprobación. 128+1920+63 488+1 048 576 = 1 114 112 = `110000`, el número total de puntos de código. Casi todos los caracteres de uso frecuente están, sin embargo, en las tres primeras filas.

Solución del ejercicio 1.6

Datos. las filas del cuadro 1.2 sin la condición de usar la fila más corta.

Planteamiento. (a) exhibir dos cadenas de bytes distintas que se decodifiquen como el mismo carácter; (b) calcular el valor máximo que admite un byte inicial `C0` o `C1`.

Desarrollo.

1. (a) «a» = `61` tiene, con 11 bits, la escritura `00001100001`. En la fila de 2 bytes daría `110 00001` = `C1` y `10 100001` = `A1`.
2. Las cadenas (`61`) y (`C1`, `A1`) son distintas y se decodificarían ambas como «a»: la decodificación no sería inyectiva.
3. Para un tokenizador de bytes, un mismo texto tendría varias representaciones con tokens distintos, y el modelo vería como diferentes dos entradas iguales. Además, un filtro que busque un carácter por sus bytes podría ser burlado con la forma sobrelarga.
4. (b) `C0` = `11000000` y `C1` = `11000001` aportan los 5 bits útiles `0000x`. El valor de 11 bits resultante es como mucho `00001111111` = 127.
5. Todo valor ≤ 127 cabe en la fila de 1 byte, luego una cadena que empiece por `C0` o `C1` es necesariamente sobrelarga.

Resultado. **(a)** (`61`) **y** (`C1`, `A1`) **representarían ambas «a»; (b** `C0` **y** `C1` **solo codifican valores menores que 128.**

Comprobación. el byte inicial más pequeño válido de la fila de 2 bytes es `C2`, que corresponde a `80` = 128, el primer punto de código de esa fila.

A.2 Tema 2

Solución del ejercicio 2.1

Datos. BPE a nivel de byte; $\mathcal{V}_0 = \mathcal{B}$.

Planteamiento. basta ver que todo texto se escribe con los símbolos de $\mathcal{V}_0$ y que las fusiones no los eliminan.

Desarrollo.

1. Todo texto, en cualquier idioma, se codifica en UTF-8 como una cadena de bytes (corolario 1.10).
2. Un byte tiene 8 bits y toma $2^8 = 256$ valores, todos en $\mathcal{V}_0$.
3. Las fusiones solo añaden símbolos (proposición 2.4); en el peor caso no se aplica ninguna y cada byte es un token (corolario 2.12).

Resultado. **no hay tokens desconocidos porque los 256 bytes están en el vocabulario; el vocabulario inicial mide 256.**

Comprobación. la frase rusa de la tema 3, con 22 bytes, da 22 tokens con el tokenizador de la práctica: ninguna fusión aplica y aun así se codifica.

Solución del ejercicio 2.2

Datos. (a) n símbolos iniciales y objetivo m; (b) 256 bytes, 1 token especial y $K = 50\,257$.

Planteamiento. cada fusión añade un símbolo (proposición 2.4).

Desarrollo.

1. (a) $n + k = m$, luego $k = m - n$.
2. (b) El vocabulario que no procede de fusiones tiene $256 + 1 = 257$ símbolos, luego $k = 50\,257 - 257 = 50\,000$.

Resultado. **(a)** $m - n$ **fusiones; (b)** $50\,000$ **fusiones.**

Comprobación. $256 + 50\,000 + 1 = 50\,257$. El número redondo indica que $k = 50\,000$ se fijó como hiperparámetro.

Solución del ejercicio 2.3

Datos. «rana» ×1, «lana» ×2, «ana» ×1, «plan» ×1; 3 fusiones.

Planteamiento. contar pares ponderados por la frecuencia, fusionar el máximo y restar su frecuencia (proposición 2.6). No hay empates en cabeza.

Desarrollo.

1. Símbolos iniciales: $4 \cdot 1 + 4 \cdot 2 + 3 \cdot 1 + 4 \cdot 1 = 4 + 8 + 3 + 4 = 19$.
2. Paso 1: $N(\texttt{a},\texttt{n}) = 1 + 2 + 1 + 1 = 5$; $N(\texttt{n},\texttt{a}) = 1 + 2 + 1 = 4$; $N(\texttt{l},\texttt{a}) = 2 + 1 = 3$; $N(\texttt{r},\texttt{a}) = N(\texttt{p},\texttt{l}) = 1$. Se fusiona (a, n): r·an·a; l·an·a×2; an·a; p·l·an, con $19 - 5 = 14$ símbolos.
3. Paso 2: $N(\texttt{an},\texttt{a}) = 1 + 2 + 1 = 4$; $N(\texttt{l},\texttt{an}) = 2 + 1 = 3$; $N(\texttt{r},\texttt{an}) = N(\texttt{p},\texttt{l}) = 1$. Se fusiona (an, a): r·ana; l·ana×2; ana; p·l·an, con $14 - 4 = 10$ símbolos.
4. Paso 3: $N(\texttt{l},\texttt{ana}) = 2$; $N(\texttt{r},\texttt{ana}) = N(\texttt{p},\texttt{l}) = N(\texttt{l},\texttt{an}) = 1$. Se fusiona (l, ana): r·ana; lana×2; ana; p·l·an, con $10 - 2 = 8$ símbolos.

Resultado. **fusiones** a + n **(5),** an + a **(4) y** l + ana **(2); símbolos** $19 \to 14 \to 10 \to 8$**.**

Comprobación. recuento directo del corpus final: $2 + 1 \cdot 2 + 1 + 3 = 8$.

Solución del ejercicio 2.4

Datos. fusiones en orden: (1) a + s, (2) c + as, (3) cas + a, (4) cas + as.

Planteamiento. aplicar las fusiones una a una, en el orden aprendido (definición 2.9).

Desarrollo.

1. «masas» = `m · a · s · a · s`. (1) da `m · as · as`; (2) no aplica porque no hay `c`; (3) y (4) no aplican porque no hay `cas`. Resultado: `m · as · as`.
2. «casa» = `c · a · s · a`. (1) da `c · as · a`; (2), `cas · a`; (3), `casa`; (4) no aplica. Resultado: un token.
3. «cascas» = `c · a · s · c · a · s`. (1) da `c · as · c · as`; (2), `cas · cas`; (3) no aplica porque tras `cas` viene `cas` y no `a`; (4) tampoco, porque tras `cas` no viene `as`. Resultado: `cas · cas`.

Resultado. **«masas» → `m · as · as` (3 tokens); «casa» → `casa` (1); «cascas» → `cas · cas` (2).**

Comprobación. al concatenar se recupera cada palabra, como garantiza el teorema 2.11.

Solución del ejercicio 2.5

Datos. «paso» ×4, «pasos» ×2, «piso» ×3, «pisos» ×3; desempate por el par lexicográficamente mayor.

Planteamiento. como en el ejemplo 2.8, aplicando la regla de desempate cuando haga falta.

Desarrollo.

1. Símbolos iniciales: $4 \cdot 4 + 2 \cdot 5 + 3 \cdot 4 + 3 \cdot 5 = 16 + 10 + 12 + 15 = 53$.
2. Paso 1: $N(\texttt{s},\texttt{o}) = 4 + 2 + 3 + 3 = 12$; $N(\texttt{p},\texttt{a}) = N(\texttt{a},\texttt{s}) = N(\texttt{p},\texttt{i}) = N(\texttt{i},\texttt{s}) = 6$; $N(\texttt{o},\texttt{s}) = 2 + 3 = 5$. Sin empate: se fusiona $(\texttt{s},\texttt{o})$ y quedan $53 - 12 = 41$ símbolos.
3. Paso 2: $(\texttt{p},\texttt{a})$, $(\texttt{a},\texttt{so})$, $(\texttt{p},\texttt{i})$ e $(\texttt{i},\texttt{so})$ empatan a 6; $(\texttt{so},\texttt{s})$ tiene 5. El primer símbolo mayor es `p`; entre $(\texttt{p},\texttt{a})$ y $(\texttt{p},\texttt{i})$ gana `i`. Se fusiona $(\texttt{p},\texttt{i})$: $41 - 6 = 35$.
4. Paso 3: $(\texttt{p},\texttt{a})$, $(\texttt{a},\texttt{so})$ y $(\texttt{pi},\texttt{so})$ empatan a 6. Como $\texttt{pi} > \texttt{p} > \texttt{a}$, se fusiona $(\texttt{pi},\texttt{so})$: $35 - 6 = 29$.
5. Paso 4: $(\texttt{p},\texttt{a})$ y $(\texttt{a},\texttt{so})$ empatan a 6; $(\texttt{piso},\texttt{s})$ tiene 3 y $(\texttt{so},\texttt{s})$, 2. Gana $(\texttt{p},\texttt{a})$ porque $\texttt{p} > \texttt{a}$: $29 - 6 = 23$.
6. (b) Fusiones en orden: `so`, `pi`, `piso`, `pa`. «peso» = `p · e · s · o` → `p · e · so`. «sopa» = `s · o · p · a` → `so · p · a` → `so · pa`. «oso» = `o · s · o` → `o · so`.

Resultado. **fusiones `s + o` (12), `p + i` (6), `pi + so` (6) y `p + a` (6); símbolos** $53 \to 41 \to 35 \to 29 \to 23$**; «peso» → `p · e · so`, «sopa» → `so · pa`, «oso» → `o · so`.**

Comprobación. recuento del corpus final, `pa·so`×4, `pa·so·s`×2, `piso`×3, `piso·s`×3: $8 + 6 + 3 + 6 = 23$. Con la regla «el primer par que aparece», el paso 2 habría elegido $(\texttt{p},\texttt{a})$ y el tokenizador sería otro.

Solución del ejercicio 2.6

Datos. secuencia de longitud L; par (a, b) con $a \neq b$ y c apariciones en las posiciones $i_1 < \cdots < i_c$ (cada i_r es la posición de la a).

Planteamiento. probar que $i_{r+1} \geq i_r + 2$, de modo que las sustituciones no comparten símbolos, y contar.

Desarrollo.

1. Si $i_{r+1} = i_r + 1$, el símbolo de la posición $i_r + 1$ sería b (segundo del par r) y a (primero del par $r + 1$), luego $a = b$: contradicción. Así $i_{r+1} \geq i_r + 2$.
2. Los intervalos $\{i_r, i_r + 1\}$ son, por tanto, disjuntos, y la fusión de izquierda a derecha sustituye los c.
3. Cada sustitución cambia 2 símbolos por 1 y resta 1; en total, $L' = L - c$.
4. Contraejemplo con $a = b$: «`aaa`» tiene $c = 2$ pares (`a`, `a`) que se solapan; la fusión da `aa`·`a`, de longitud $2 = 3 - 1 \neq 3 - 2$.
5. «`aaaa`» tiene 3 pares; la fusión da `aa`·`aa`, de longitud $2 = 4 - 2$. «`aaaaa`» tiene 4 pares; la fusión da `aa`·`aa`·`a`, de longitud $3 = 5 - 2$.

Resultado. $L' = L - c$ **si** $a \neq b$; con $a = b$ solo cuentan las apariciones no solapadas, $\lfloor r/2 \rfloor$ en cada racha de r símbolos iguales: longitudes 2 y 3 para «`aaaa`» y «`aaaaa`».

Comprobación. en el cuadro 2.1, $30 - 8 = 22$, $22 - 5 = 17$, $17 - 3 = 14$ y $14 - 2 = 12$ coinciden con el recuento directo.

Solución del ejercicio 2.7

Datos. tokenizador BPE de bytes con el token `C3`.

Planteamiento. usar el cuadro 1.2 para (a), el teorema 2.11 y el teorema 1.7 para (b).

Desarrollo.

1. (a) `C3` = `11000011` anuncia un carácter de 2 bytes; aislado, falta el byte de continuación y la cadena no es UTF-8 válida. La decodificación estricta da error y la tolerante devuelve el carácter de sustitución `U+FFFD`.
2. (b) Sea $b = \mathrm{enc}(x)$ la cadena UTF-8 del texto x y sea $(t_1, \ldots, t_m)$ su tokenización. Por el teorema 2.11 con $\mathcal{V}_0 = \mathcal{B}$, $t_1 \| \ldots \| t_m = b$.
3. Por el teorema 1.7, enc es inyectiva y b es una cadena válida, luego $\mathrm{enc}^{-1}(t_1 \| \ldots \| t_m) = \mathrm{enc}^{-1}(b) = x$.
4. (c) Si la «ñ» = `C3 B1` llega en dos tokens y cada uno se decodifica al recibirlo, se muestran dos caracteres `U+FFFD` en lugar de «ñ».
5. Remedio: acumular en un búfer los bytes de un carácter incompleto (se sabe que lo está por su byte inicial, paso 1 de la demostración del teorema 1.7) y mostrarlo cuando llegue su último byte.

Resultado. **(a) error o `U+FFFD`; (b) la concatenación de los tokens es la cadena UTF-8 original, que se decodifica sin pérdida; (c) el usuario ve dos `U+FFFD`; se corrige con un búfer de bytes.**

Comprobación. en Python, decodificar por separado los dos bytes de `"ñ".encode()` con `errors="replace"` da dos `U+FFFD`, mientras que `"ñ".encode().decode()` devuelve «ñ».

A.3 Tema 3

Solución del ejercicio 3.1

Datos. un LLM multilingüe con vocabulario de palabras completas.

Planteamiento. comparar lo que exige ese vocabulario con lo que necesita el modelo: un K manejable y capacidad para representar cualquier texto.

Desarrollo.

1. Tamaño: cada forma flexionada («canto», «cantas», «cantaré»…) es un token, y con decenas de idiomas K supera con creces el medio millón. Por la proposición 3.6, las matrices de entrada y salida crecen en proporción.
2. Cobertura: una palabra nueva, una errata o un nombre propio raro no estarían en $\mathcal{V}$; habría que sustituirlos por un token «desconocido» y se perdería su información.
3. Además, «caballería» y «caballerías» tendrían filas independientes en E, y las formas raras recibirían muy pocos ejemplos de entrenamiento.

Resultado. **vocabulario gigantesco (matrices de embeddings y de salida enormes) y palabras fuera del vocabulario sin representación**; además, las formas emparentadas no comparten parámetros.

Comprobación. las subpalabras resuelven los tres problemas: K entre $3{,}2 \cdot 10^4$ y $2{,}6 \cdot 10^5$ y toda palabra se descompone en piezas conocidas (corolario 2.12).

Solución del ejercicio 3.2

Datos. «ferrocarril» tiene 11 letras, 4 de ellas «r».

Planteamiento. el modelo recibe tokens, no letras; hay que ver qué información falta en la entrada.

Desarrollo.

1. El tokenizador divide la palabra en pocas piezas, por ejemplo `ferro` · `carril`, y el modelo recibe dos enteros.
2. Cada entero se convierte en una fila de E, que no contiene una lista explícita de letras.
3. Para contar, el modelo debe saber que `ferro` contiene 2 «r» y `carril` otras 2, y sumarlas. Esa ortografía solo se aprende de forma indirecta, de textos que deletrean palabras.
4. Remedio: escribir la palabra separada letra a letra; entonces cada letra es, casi siempre, un token.

Resultado. **falla porque no ve letras sino tokens; contar exige conocer la ortografía interna de cada token, que no está en la entrada.**

Comprobación. con la palabra deletreada, el número de tokens que son «r» coincide con la respuesta, 4.

Solución del ejercicio 3.3

Planteamiento. contrastar cada afirmación con la definición 2.9, la observación 1.1 y la proposición 3.6.

Desarrollo.

(a) **Falsa.** BPE aplica las fusiones en un orden fijo y es determinista; el modelo unigram es el que admite varias segmentaciones y permite muestrearlas.

(b) **Falsa.** El tokenizador se entrena antes, con un algoritmo de conteo, y se congela.

(c) **Verdadera.** La matriz tiene Kd parámetros; con d fijo, $(2K)d = 2(Kd)$.

Resultado. **(a) falsa; (b) falsa; (c) verdadera.**

Comprobación. en (c), con $d = 4096$: $32\,000 \cdot 4096 = 131\,072\,000$ y $64\,000 \cdot 4096 = 262\,144\,000$, exactamente el doble.

Solución del ejercicio 3.4

Datos. vocabulario del ejemplo 3.2; $s^* = (\texttt{casa}, \texttt{s})$.

Planteamiento. $-\ln P(s) = -\sum_i \ln p(t_i)$.

Desarrollo.

1. $P(s^*) = 0{,}2 \cdot 0{,}1 = 0{,}02$ y $-\ln 0{,}02 = \ln 100 - \ln 2 = 4{,}6052 - 0{,}6931 = 3{,}912$ nats.
2. $P(\texttt{cas}, \texttt{as}) = 0{,}1 \cdot 0{,}15 = 0{,}015$ y $-\ln 0{,}015 = \ln 100 - \ln 1{,}5 = 4{,}6052 - 0{,}4055 = 4{,}200$ nats.
3. Diferencia: $\ln(0{,}02/0{,}015) = \ln(4/3) = 0{,}2877$ nats.

Resultado. $-\ln P(s^*) = 3{,}912$ **nats; con** $(\texttt{cas}, \texttt{as})$ **se pierden** $\ln(4/3) \approx 0{,}2877$ **nats.**

Comprobación. la diferencia es positiva porque s^* es la segmentación de máxima probabilidad. Restar valores ya redondeados, $4{,}2 - 3{,}9 = 0{,}3$, daría una cifra demasiado gruesa.

Solución del ejercicio 3.5

Datos. $K = 32\,000$, $d = 4096$, matrices separadas, 2 bytes por parámetro.

Planteamiento. cada matriz tiene Kd parámetros (proposición 3.6).

Desarrollo.

1. Una matriz: $32\,000 \cdot 4096 = 131\,072\,000$.
2. Dos matrices: $2 \cdot 131\,072\,000 = 262\,144\,000$.
3. Memoria: $262\,144\,000 \cdot 2 = 524\,288\,000$ bytes $= 0{,}5243$ GB (10^9 bytes), o $0{,}4883$ GiB (2^{30} bytes).

Resultado. $262\,144\,000$ **parámetros, que ocupan unos** $0{,}52$ **GB en** `bf16`**.**

Comprobación. coincide con la fila de Llama 2 7B del cuadro 3.3: 131 millones por matriz.

Solución del ejercicio 3.6

Datos. agrupación de los dígitos en bloques de hasta 3, de izquierda a derecha.

Planteamiento. aplicar la regla a cada número y observar a qué posición decimal corresponde cada dígito dentro de su token.

Desarrollo.

1. (a) $1605 \to \texttt{160} \cdot \texttt{5}$; $2026 \to \texttt{202} \cdot \texttt{6}$; $123456 \to \texttt{123} \cdot \texttt{456}$; $1234567 \to \texttt{123} \cdot \texttt{456} \cdot \texttt{7}$.
2. (b) $999 \to \texttt{999}$, un token; $999 + 1 = 1000 \to \texttt{100} \cdot \texttt{0}$, dos tokens que no tienen ningún bloque en común con el anterior.
3. (c) Con la agrupación de izquierda a derecha, el último token de 1605 son las unidades, pero el último de 123456 son centenas, decenas y unidades: la posición decimal de un dígito depende de la longitud del número, y el acarreo de una suma puede cambiar la segmentación entera.

4. Una agrupación de derecha a izquierda, como la de los separadores de millares, alinea los bloques con las potencias de 10^3: 1605 → `1 · 605`, 1234567 → `1 · 234 · 567`.

Resultado. **(a) `160·5`, `202·6`, `123·456`, `123·456·7`; (b) 1 token frente a 2; (c) la agrupación de izquierda a derecha desalinea las posiciones decimales; la de derecha a izquierda las respeta.**

Comprobación. con la agrupación de derecha a izquierda, 999 → `999` y 1000 → `1 · 000`: el bloque de unidades, decenas y centenas ocupa siempre el último token.

Solución del ejercicio 3.7

Datos. cadena de n caracteres; vocabulario con todos los caracteres y tokens de longitud a lo sumo ℓ.

Planteamiento. (a) biyección entre segmentaciones y conjuntos de cortes; (b) asociar a cada segmentación la composición de n dada por las longitudes de sus tokens.

Desarrollo.

1. (a) Una segmentación en subcadenas no vacías queda determinada por el conjunto de los huecos entre caracteres consecutivos en los que se corta. Hay $n-1$ huecos y cada uno se corta o no, de forma independiente: 2^{n-1} segmentaciones.
2. (b) A cada segmentación con tokens de $\mathcal{V}$ se le asocia la secuencia de longitudes $(\ell_1, \ldots, \ell_m)$, con $\ell_i \in \{1, \ldots, \ell\}$ y $\sum \ell_i = n$. La aplicación es inyectiva, porque las longitudes determinan los puntos de corte; luego el número de segmentaciones es a lo sumo el número $c_\ell(n)$ de composiciones de n con partes en $\{1, \ldots, \ell\}$.
3. c_ℓ satisface $c_\ell(0) = 1$ y $c_\ell(n) = \sum_{j=1}^{\min(\ell,n)} c_\ell(n-j)$, porque la última parte vale j. Con $\ell = 2$ es la sucesión de Fibonacci, $c_2(n) = F_{n+1}$, que crece exponencialmente; por ejemplo, $c_2(20) = F_{21} = 10\,946$.
4. Si $\mathcal{V}$ contiene todas las subcadenas de longitud a lo sumo ℓ, la cota se alcanza, y enumerar las segmentaciones cuesta un tiempo exponencial en n. El teorema 3.3 encuentra la mejor con $O(n\ell)$ productos.

Resultado. **(a) 2^{n-1}; (b) a lo sumo $c_\ell(n)$, que crece exponencialmente, frente al coste lineal $O(n\ell)$ de Viterbi.**

Comprobación. con $n = 5$ y $\ell = 5$, $c_5(5) = 2^4 = 16$ composiciones; con el vocabulario del ejemplo 3.2, solo 7 de las 16 segmentaciones de «casas» usan tokens de $\mathcal{V}$ (cuadro 3.1).

A.4 Tema 4

Solución del ejercicio 4.1

Datos. E de tamaño 3×2; $e_1 = (0, 1, 0)^\top$.

Planteamiento. calcular el producto componente a componente y contar las multiplicaciones de un producto $e_i^\top E$ general.

Desarrollo.

1. Primera componente: $0 \cdot 1 + 1 \cdot 2 + 0 \cdot 0 = 2$. Segunda: $0 \cdot 0 + 1 \cdot (-1) + 0 \cdot 3 = -1$.
2. Luego $e_1^\top E = (2, -1) = E_{1,:}$, la fila de índice 1.

3. El producto general requiere K multiplicaciones por cada una de las d componentes: $Kd = 50\,257 \cdot 768 = 38\,597\,376$.

Resultado. $e_1^\top E = (2, -1)$**; el producto completo costaría** $38\,597\,376$ **multiplicaciones**, frente a ninguna con la indexación.

Comprobación. la cifra coincide con el número de parámetros de la matriz de GPT-2 pequeño (cuadro 3.3), como debe ser: hay una multiplicación por entrada de E.

Solución del ejercicio 4.2

Datos. «la luna brilla», $n = 3$; ventanas $m = 1$ y $m = 2$.

Planteamiento. emparejar cada palabra con las que están a distancia a lo sumo m.

Desarrollo.

1. $m = 1$: (la, luna); (luna, la), (luna, brilla); (brilla, luna). Total: $1 + 2 + 1 = 4$.
2. $m = 2$: se añaden (la, brilla) y (brilla, la). Total: 6.
3. proposición 4.7: $1 \cdot (6 - 1 - 1) = 4$ y $2 \cdot (6 - 2 - 1) = 6$; se cumple $n \geq m + 1$ en ambos casos.

Resultado. **4 pares con ventana 1 y 6 con ventana 2.**

Comprobación. con $m = 2$ cada palabra se empareja con las otras dos: $3 \cdot 2 = 6$ pares ordenados.

Solución del ejercicio 4.3

Datos. cereza $= (40, 3)$, fresa $= (20, 4)$, digital $= (2, 30)$.

Planteamiento. $\cos(a, b) = a \cdot b / (\|a\| \|b\|)$.

Desarrollo.

1. Normas: $\|\text{cereza}\| = \sqrt{1609} = 40{,}11$; $\|\text{fresa}\| = \sqrt{416} = 20{,}40$; $\|\text{digital}\| = \sqrt{904} = 30{,}07$.
2. cereza $\cdot$ fresa $= 800 + 12 = 812$ y $\sqrt{1609 \cdot 416} = \sqrt{669\,344} = 818{,}1$; coseno $812/818{,}1 = 0{,}9925$.
3. cereza $\cdot$ digital $= 80 + 90 = 170$ y $\sqrt{1609 \cdot 904} = \sqrt{1\,454\,536} = 1206$; coseno $170/1206 = 0{,}1410$.
4. (b) Para $\lambda > 0$: $(\lambda a) \cdot b = \lambda(a \cdot b)$ y $\|\lambda a\| = |\lambda| \|a\| = \lambda \|a\|$; luego $\cos(\lambda a, b) = \lambda(a \cdot b)/(\lambda \|a\| \|b\|) = \cos(a, b)$.
5. La propiedad es deseable porque la longitud de un vector de recuentos crece con la frecuencia de la palabra, y la similitud no debe depender de cuán frecuente sea cada palabra.

Resultado. $\cos(\textbf{cereza}, \textbf{fresa}) \approx 0{,}9925$ **y** $\cos(\textbf{cereza}, \textbf{digital}) \approx 0{,}1410$**: cereza se parece mucho más a fresa.**

Comprobación. $3 \cdot \text{cereza} = (120, 9)$ da el mismo coseno con fresa: $(2400 + 36)/\sqrt{14\,481 \cdot 416} = 2436/2454 = 0{,}9925$.

Solución del ejercicio 4.4

Datos. $N(\text{gato}, \text{come}) = N(\text{gato}, \text{maúlla}) = N(\text{perro}, \text{come}) = N(\text{perro}, \text{ladra}) = 4$; el resto, 0; $k = 2$.

Planteamiento. calcular las probabilidades conjuntas y marginales y aplicar la definición 4.5 y el teorema 4.13.

Desarrollo.

1. Total: $|\mathcal{D}| = 16$. Marginales: $P(\text{gato}) = P(\text{perro}) = 8/16 = 1/2$; $P(\text{come}) = 8/16 = 1/2$; $P(\text{maúlla}) = P(\text{ladra}) = 4/16 = 1/4$.
2. $\text{PMI}(\text{gato}, \text{come}) = \ln \dfrac{4/16}{(1/2)(1/2)} = \ln 1 = 0$.
3. $\text{PMI}(\text{gato}, \text{maúlla}) = \ln \dfrac{4/16}{(1/2)(1/4)} = \ln 2 = 0{,}6931$.
4. $\text{PMI}(\text{gato}, \text{ladra}) = \ln 0 = -\infty$, luego PPMI $= 0$. Por simetría, «perro» da 0 con «come», $\ln 2$ con «ladra» y $-\infty$ con «maúlla».
5. (b) Para los pares observados, $u_c \cdot v_w = \text{PMI} - \ln 2$: $0 - 0{,}6931 = -0{,}6931$ con «come» y $0{,}6931 - 0{,}6931 = 0$ con «maúlla» o «ladra».

Resultado. PMI $= 0$ **con «come»,** $\ln 2 \approx 0{,}6931$ **con el verbo propio de cada animal y** $-\infty$ **(**PPMI $= 0$**) con el ajeno; SGNS con** $k = 2$ **aprendería productos** $-0{,}6931$ **y** 0**.**

Comprobación. «come» acompaña a ambos animales con la misma frecuencia, de modo que no informa: su PMI es nula, mientras que «maúlla» duplica la frecuencia esperada con «gato» ($e^{\ln 2} = 2$).

Solución del ejercicio 4.5

Datos. $J = -\ln \hat{y}_o$ con $\hat{y} = \text{softmax}(U^\top v_c)$; $u_{\text{come}} = (0, 1)$, $u_{\text{avión}} = (-1, 0)$, $v_c = (1, 0)$, $o = \text{come}$.

Planteamiento. partir de la proposición 4.9 y derivar $J = -u_o^\top v_c + \ln \sum_w \exp(u_w^\top v_c)$ respecto a cada u_w.

Desarrollo.

1. (a) $\sum_w \hat{y}_w u_w = U\hat{y}$ y $u_o = Uy$, luego $\partial J/\partial v_c = U(\hat{y} - y)$. Se anula si y solo si $u_o = \sum_w \hat{y}_w u_w$, es decir, si el vector de contexto real coincide con la media de los vectores de contexto ponderada por la predicción.
2. (b) El término $-u_o^\top v_c$ solo depende de u_o y da $-v_c$; el segundo da $\hat{y}_w v_c$. Luego $\partial J/\partial u_o = (\hat{y}_o - 1)v_c$ y $\partial J/\partial u_w = \hat{y}_w v_c$ si $w \neq o$; en una sola fórmula, $(\hat{y}_w - y_w)v_c$.
3. (c) Puntuaciones: $u_{\text{come}} \cdot v_c = 0$ y $u_{\text{avión}} \cdot v_c = -1$. Softmax: $\hat{y} = (e^0, e^{-1})/(1 + e^{-1}) = (0{,}7311;\ 0{,}2689)$.
4. $\partial J/\partial v_c = 0{,}7311\,(0, 1) + 0{,}2689\,(-1, 0) - (0, 1) = (-0{,}2689;\ -0{,}2689)$.
5. $\partial J/\partial u_{\text{come}} = (0{,}7311 - 1)(1, 0) = (-0{,}2689;\ 0)$ y $\partial J/\partial u_{\text{avión}} = 0{,}2689\,(1, 0) = (0{,}2689;\ 0)$.

Resultado. $\partial J/\partial v_c = U(\hat{y} - y) = (-0{,}2689; -0{,}2689)$**;** $\partial J/\partial u_w = (\hat{y}_w - y_w)v_c$**, que vale** $(-0{,}2689; 0)$ **para «come» y** $(0{,}2689; 0)$ **para «avión».**

Comprobación. la pérdida inicial es $-\ln 0{,}7311 = 0{,}3133$. Un paso con $\eta = 0{,}1$ sobre v_c da $v_c' = (1{,}0269;\ 0{,}0269)$, puntuaciones $0{,}0269$ y $-1{,}0269$, y pérdida $0{,}2991 < 0{,}3133$.

Solución del ejercicio 4.6

Datos. $L = -\ln \sigma(c_+ \cdot w) - \ln \sigma(-c_- \cdot w)$; $w = (1, 0)$, $c_+ = (0, 1)$, $c_- = (-1, 0)$; $\eta = 0{,}1$.

Planteamiento. regla de la cadena con $\sigma'(x) = \sigma(x)(1-\sigma(x))$, de donde $\frac{d}{dx}[-\ln\sigma(x)] = \sigma(x)-1$, y $1-\sigma(-x) = \sigma(x)$.

Desarrollo.

1. Término positivo, con $s_+ = c_+ \cdot w$: su derivada respecto a s_+ es $\sigma(s_+)-1$, y $\partial s_+/\partial c_+ = w$. Luego $\partial L/\partial c_+ = (\sigma(c_+ \cdot w) - 1)\, w$.
2. Término negativo, con $s_- = -c_- \cdot w$: su derivada es $\sigma(s_-)-1 = -\sigma(c_- \cdot w)$, y $\partial s_-/\partial c_- = -w$. Luego $\partial L/\partial c_- = \sigma(c_- \cdot w)\, w$.
3. Respecto a w intervienen ambos términos: $\partial L/\partial w = (\sigma(c_+ \cdot w) - 1)\, c_+ + \sigma(c_- \cdot w)\, c_-$.
4. (b) $c_+ \cdot w = 0$ y $c_- \cdot w = -1$; $\sigma(0) = 0{,}5$ y $\sigma(-1) = 1/(1+e) = 0{,}2689$.
5. $\partial L/\partial c_+ = -0{,}5\,(1,0) = (-0{,}5;\ 0)$; $\partial L/\partial c_- = 0{,}2689\,(1,0) = (0{,}2689;\ 0)$; $\partial L/\partial w = -0{,}5\,(0,1) + 0{,}2689\,(-1,0) = (-0{,}2689;\ -0{,}5)$.
6. (c) $w' = w - \eta\,\partial L/\partial w = (1 + 0{,}02689;\ 0 + 0{,}05) = (1{,}027;\ 0{,}05)$.
7. Nueva pérdida: $c_+ \cdot w' = 0{,}05$ da $-\ln\sigma(0{,}05) = 0{,}6685$; $c_- \cdot w' = -1{,}027$ da $-\ln\sigma(1{,}027) = 0{,}3061$. $L' = 0{,}9746$.

Resultado. $\partial L/\partial c_+ = (-0{,}5;\ 0)$**,** $\partial L/\partial c_- = (0{,}2689;\ 0)$**,** $\partial L/\partial w = (-0{,}2689;\ -0{,}5)$**; tras un paso,** L **baja de** $1{,}006$ **a** $0{,}9746$**.**

Comprobación. el paso acerca w a c_+ (su segunda componente sube de 0 a 0,05) y lo aleja de c_-, que apunta en el sentido negativo del primer eje. Un gradiente numérico por diferencias centradas da los mismos vectores.

A.5 Tema 5

Solución del ejercicio 5.1

Datos. $N = 440$ palabras, $K = 243$ tipos, 416 bigramas distintos, 398 únicos.

Planteamiento. aplicar la proposición 5.1 con $n = 2$ y recordar el EMV del módulo 1.

Desarrollo.

1. (a) Cota: $N - n + 1 = 439$ bigramas distintos como máximo, y se observan $416 \le 439$. Fracción máxima: $439/243^2 = 439/59\,049 = 0{,}743\,\%$; la observada es $416/59\,049 = 0{,}704\,\%$.
2. (b) $398/416 = 0{,}9567$: el 95,67 % de los bigramas observados aparece una sola vez.
3. (c) El EMV asigna probabilidad 0 a todo bigrama no observado. Si el texto de evaluación contiene uno, su probabilidad es 0, la pérdida es infinita y la perplejidad también (módulo 1).

Resultado. **(a)** $416 \le 439$ **y** $0{,}704\,\% \le 0{,}743\,\%$**; (b)** $95{,}67\,\%$**; (c) probabilidad nula y perplejidad infinita.**

Comprobación. casi todos los recuentos positivos valen 1, de modo que la estimación de las pocas casillas no vacías tampoco es fiable: la dispersión no se arregla solo con suavizado.

Solución del ejercicio 5.2

Datos. $g = (3, -4, 12)$, $\tau = 5$.

Planteamiento. calcular $\|g\|$ y, como supera τ, reescalar.

Desarrollo.

1. $\|g\| = \sqrt{9+16+144} = \sqrt{169} = 13 \geq 5$.
2. $\text{clip}_5(g) = \frac{5}{13}(3,-4,12) = (15/13,\ -20/13,\ 60/13) = (1{,}154;\ -1{,}538;\ 4{,}615)$.
3. Norma: $\frac{5}{13} \cdot 13 = 5$.
4. El vector es λg con $\lambda = 5/13 > 0$: misma dirección y sentido.

Resultado. $\text{clip}_5(g) = (15/13,\ -20/13,\ 60/13) \approx (1{,}154;\ -1{,}538;\ 4{,}615)$**, de norma 5.**

Comprobación. $(15^2 + 20^2 + 60^2)/13^2 = (225 + 400 + 3600)/169 = 4225/169 = 25$, y $\sqrt{25} = 5$.

Solución del ejercicio 5.3

Datos. f constante; 100 pasos; RNN escalar con $w = 0{,}9$.

Planteamiento. por la proposición 5.11, el factor del camino directo es f^{100}.

Desarrollo.

1. (a) $0{,}98^{100} = e^{100 \ln 0{,}98} = e^{-2{,}020} = 0{,}1326$. En la RNN, $0{,}9^{100} = 2{,}656 \cdot 10^{-5}$ (cuadro 5.1).
2. Cociente: $0{,}1326/2{,}656 \cdot 10^{-5} = 4993 \approx 5{,}0 \cdot 10^3$.
3. (b) $f^{100} \geq 0{,}5 \iff 100 \ln f \geq \ln 0{,}5 = -0{,}6931 \iff f \geq e^{-0{,}006931} = 0{,}993092$.
4. Para dar un valor seguro con 4 cifras decimales se redondea hacia arriba: $f \geq 0{,}9931$.

Resultado. **(a)** $0{,}1326$**, unas** $5 \cdot 10^3$ **veces más que en la RNN; (b)** $f \geq 0{,}5^{1/100} \approx 0{,}99309$**.**

Comprobación. $0{,}9931^{100} = 0{,}5004 \geq 0{,}5$, mientras que $0{,}9930^{100} = 0{,}4954 < 0{,}5$.

Solución del ejercicio 5.4

Datos. $h = d = 256$; la GRU tiene tres transformaciones como las de la RNN.

Planteamiento. proposición 5.4 y corolario 5.12.

Desarrollo.

1. RNN: $h(h+d) + h = 256 \cdot 512 + 256 = 131\,072 + 256 = 131\,328$.
2. LSTM: $4 \cdot 131\,328 = 525\,312$.
3. GRU: $3 \cdot 131\,328 = 393\,984$.
4. Cociente LSTM/RNN: $525\,312/131\,328 = 4$.

Resultado. **RNN** $131\,328$**, GRU** $393\,984$ **y LSTM** $525\,312$ **parámetros; un paso de LSTM cuesta unas 4 veces más que uno de RNN.**

Comprobación. los tres modelos siguen siendo secuenciales: el factor 4 aumenta el coste de cada paso, pero no cambia el número de pasos, T.

Solución del ejercicio 5.5

Datos. $h_t = w\, h_{t-1}$, $0 < w < 1$; factor a 100 pasos $w^{100} \geq 0{,}1$.

Planteamiento. tomar logaritmos; ln es creciente y conserva la desigualdad.

Desarrollo.

1. $100 \ln w \geq \ln 0{,}1 = -2{,}3026$.
2. $\ln w \geq -0{,}023026$, luego $w \geq e^{-0{,}023026} = 0{,}977237$.
3. Con 4 cifras decimales, redondear hacia abajo no basta: $0{,}9772^{100} = 0{,}0996 < 0{,}1$, mientras que $0{,}9773^{100} = 0{,}1006$. El valor seguro es 0,9773.
4. Interpretación: con $w < 0{,}9772$ llega menos del 10 % de la señal a 100 pasos, y con $w > 1$ el factor explota ($1{,}1^{100} \approx 13\,781$). La franja útil, entre 0,977 y 1, es estrechísima.

Resultado. $w \geq 0{,}1^{1/100} \approx 0{,}97724$ (con 4 cifras decimales, redondeando hacia arriba, 0,9773).

Comprobación. $0{,}97724^{100} = e^{100 \ln 0{,}97724} = e^{-2{,}3023} = 0{,}1000$. Las LSTM y las conexiones residuales fijan ese factor cerca de 1 por diseño (proposición 5.11).

Solución del ejercicio 5.6

Datos. M simétrica con $Mv_i = \lambda_i v_i$, $\{v_i\}$ ortonormal, $|\lambda_1| > |\lambda_2| \geq \ldots$; $x = \sum_i a_i v_i$ con $a_1 \neq 0$.

Planteamiento. (a) inducción sobre k; (b) descomponer $M^k x$ en la base de vectores propios; (c) aplicar el resultado al jacobiano $\partial h_T / \partial h_0 = M^T$.

Desarrollo.

1. (a) $M^1 v_i = \lambda_i v_i$. Si $M^{k-1} v_i = \lambda_i^{k-1} v_i$, entonces $M^k v_i = M(\lambda_i^{k-1} v_i) = \lambda_i^{k-1} \lambda_i v_i = \lambda_i^k v_i$. Como los v_i forman una base, estos son todos los valores propios de M^k.
2. (b) Por linealidad, $M^k x = a_1 \lambda_1^k v_1 + r_k$ con $r_k = \sum_{i \geq 2} a_i \lambda_i^k v_i$, que es ortogonal a v_1.
3. La tangente del ángulo θ_k entre $M^k x$ y v_1 es $\|r_k\| / |a_1 \lambda_1^k|$. Por la ortonormalidad, $\|r_k\|^2 = \sum_{i \geq 2} a_i^2 \lambda_i^{2k} \leq |\lambda_2|^{2k} \sum_{i \geq 2} a_i^2 = |\lambda_2|^{2k} \|x - a_1 v_1\|^2$.
4. Luego $\tan \theta_k \leq (|\lambda_2| / |\lambda_1|)^k \, \|x - a_1 v_1\| / |a_1|$, que tiende a 0. Para un x aleatorio con distribución continua, $a_1 \neq 0$ con probabilidad 1.
5. (c) En la RNN lineal $h_t = M h_{t-1}$, $\partial h_T / \partial h_0 = M^T$, y el gradiente que llega a h_0 es $M^T g$ con $g = \partial L / \partial h_T$. Por (b), se alinea con v_1 y su norma se comporta como $|a_1| |\lambda_1|^T$: se desvanece si $|\lambda_1| < 1$ y explota si $|\lambda_1| > 1$.

Resultado. **(a)** $M^k v_i = \lambda_i^k v_i$**; (b)** $\tan \theta_k \leq (|\lambda_2| / |\lambda_1|)^k \|x - a_1 v_1\| / |a_1| \to 0$**; (c) el gradiente de una RNN lineal decae o crece como** $|\lambda_1|^T$**.**

Comprobación. para una matriz simétrica la norma espectral es $|\lambda_1|$, de modo que (c) es coherente con la cota $\|W\|^T$ del teorema 5.6. Con $M = \mathrm{diag}(0{,}9;\ 0{,}5)$ y $x = (1, 1)$, $M^{10} x = (0{,}3487;\ 0{,}000977)$ y $\tan \theta_{10} = 0{,}0028 = (0{,}5/0{,}9)^{10}$.

Bibliografía

[1] Y. Bengio, R. Ducharme, P. Vincent y C. Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003. https://www.jmlr.org/papers/v3/bengio03a.html

[2] Y. Bengio, P. Simard y P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

[3] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 2, «Word Vectors»; clase 4, «Language Models and RNNs»; tarea 2. https://web.stanford.edu/class/cs224n/

[4] Stanford CS336. Language Modeling from Scratch. Stanford University, primavera de 2026. Tarea 1 (*basics*): tokenizador BPE. https://github.com/stanford-cs336/assignment1-basics

[5] P. Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

[6] Gemini Team, Google. Gemini: A family of highly capable multimodal models. Informe técnico, 2023. https://arxiv.org/abs/2312.11805

[7] F. A. Gers, J. Schmidhuber y F. Cummins. Learning to forget: continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000.

[8] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. Cap. 10, «Sequence Modeling: Recurrent and Recursive Nets». https://www.deeplearningbook.org/

[9] S. Hochreiter y J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[10] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 2, «Words and Tokens»; cap. 5, «Embeddings»; cap. 14, «RNNs and LSTMs». https://web.stanford.edu/~jurafsky/slp3/

[11] T. Kudo. Subword regularization: improving neural network translation models with multiple subword candidates. En *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, págs. 66–75, 2018. https://aclanthology.org/P18-1007/

[12] T. Kudo y J. Richardson. SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing. En *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, págs. 66–71, 2018. https://aclanthology.org/D18-2012/

[13] O. Levy e Y. Goldberg. Neural word embedding as implicit matrix factorization. En *Advances in Neural Information Processing Systems 27 (NIPS)*, 2014.

[14] Llama Team, AI @ Meta. The Llama 3 herd of models. Informe técnico, 2024. https://arxiv.org/abs/2407.21783

[15] T. Mikolov, K. Chen, G. Corrado y J. Dean. Efficient estimation of word representations in vector space. *arXiv:1301.3781*, 2013. https://arxiv.org/abs/1301.3781

[16] T. Mikolov, I. Sutskever, K. Chen, G. Corrado y J. Dean. Distributed representations of words and phrases and their compositionality. En *Advances in Neural Information Processing Systems 26 (NIPS)*, 2013. https://arxiv.org/abs/1310.4546

[17] R. Pascanu, T. Mikolov e Y. Bengio. On the difficulty of training recurrent neural networks. En *Proceedings of the 30th International Conference on Machine Learning (ICML)*, PMLR 28, 2013. https://proceedings.mlr.press/v28/pascanu13.html

[18] A. Petrov, E. La Malfa, P. H. S. Torr y A. Bibi. Language model tokenizers introduce unfairness between languages. En *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. https://arxiv.org/abs/2305.15425

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[20] R. Sennrich, B. Haddow y A. Birch. Neural machine translation of rare words with subword units. En *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, págs. 1715–1725, 2016. https://aclanthology.org/P16-1162/

[21] H. Touvron *et al.* Llama 2: open foundation and fine-tuned chat models. Informe técnico, 2023. https://arxiv.org/abs/2307.09288

[22] F. Yergeau. UTF-8, a transformation format of ISO 10646. RFC 3629, Internet Engineering Task Force, noviembre de 2003. https://www.rfc-editor.org/rfc/rfc3629

[23] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §9.7, §10.1 y §15.6. https://d2l.ai/